CANGJIE WORKSHOP

仓颉 AI 实践分享

分享人：刘俊杰　2026.03.28

序章：智能质变

东方盛夏 11月19日 17:15:49

最近坐飞机找到灵感，设计了这道机场航班调度算法题，1.0 版本同时考虑起降，题解过于复杂，2.0 版本简化到只规划起飞，但题解仍然要综合应用压缩 DP + 贪心 + 拆分，如果暴力求解复杂度是阶乘级，飞机数稍多就解不开，所以这个题很适合控分，暴力解能得几分，算法解得分更多，但很少有学生能联合多种算法覆盖全部场景，相信这道题能给学生留下深刻印象。

东方盛夏 11月19日 17:15:54

更值得分享的是，我前些天用很多大模型去解这个原创题（ds/kimi/qwen/gpt5/grok4，写 python），都做了很多轮对话，都没有给出完善的解法，在问题规模增大时，有的成为近似解，有的计算量暴增，和预测的学生解题情况类似。这也符合目前对 AI 的印象，简单问题还能做，稍微复杂就突破不了、原地绕圈。

东方盛夏 11月19日 17:16:59

但是，昨天听说谷歌发布了 gemini 3 pro，据说很强，我今天把这个问题抛给它，结果一次对话就给出了逆天题解，应用多种算法给出了最优解（构造了一个 5 维度状态表），又快又准确。新建对话重试一次，仍然一遍过所有用例。而且还能挤挤，我让它进一步简化优化，它还给出了代码更少的版本。感觉这个模型，已经产生智能质变了。大家之前遇到困难的问题，可以用这个模型试试。

即刻起飞

题目描述

民航客机按最大起飞重量分为轻型（L）、中型（M）和重型（H）三类机型（规定 H > M > L）。当多架客机排队在同一条跑道起飞时，为避免尾流干扰和碰撞等安全风险，通常会对相邻两架客机的起飞时间间隔做出要求，某机场规定如下：

- 若前一架机型 ≤ 后一架机型，则起飞间隔 ≥ 1 分钟
- 若前一架机型 > 后一架机型（尾流干扰更大），则起飞间隔 ≥ 2 分钟

此外，每架客机还有一个规划起飞时间，如果实际起飞时间超过规划时间，会产生 1 单位 / 分钟的等待成本。

现在有 N 架客机准备在同一条跑道起飞，请你规划一种起飞调度策略（起飞时间表），使得所有客机产生的总等待成本最小。

H3 输入

- `Array<(Rune, Int64)>`：数组元素 `(Rune, Int64)` 对应一架客机的机型符号与规划起飞时刻（分钟）

H3 输出

- `Int64`：最优调度策略对应的最小总等待成本

H3 约束

- `1 ≤ N ≤ 20`
- 从 0 时刻计时，排在第一位的飞机可以在 0 时刻起飞
- 起飞被简化为一个瞬时事件，不计时长

核心思路分析

1. **分组与贪心策略（局部有序性）：**
   - 所有飞机分为 L、M、H 三类。
   - **关键性质**：对于同一类型的飞机（例如都是 L 型），为了最小化延误成本，**一定**是按照截止时间（Deadline）从小到大的顺序起飞。
   - *证明*：假设两架同类型的飞机 A 和 B，A 的 deadline 早于 B。如果交换它们的顺序，它们对前后飞机的间隔影响不变（因为类型相同），但 A 被推迟会带来更大的潜在罚款风险。因此，每种类型的飞机内部的相对顺序是固定的。
   - 由此，我们可以将问题转化为：有三个有序队列（L队、M队、H队），每次从这三个队列的头部取出一架飞机起飞，如何决策能使总成本最小。
2. **动态规划（DP）状态定义：**
   - 我们需要记录当前三个队列分别取了多少架飞机。
   - 我们需要知道“上一架起飞的机型”，因为这决定了当前飞机的起飞时间间隔（若 前>后 则间隔2，否则间隔1）。
   - 我们需要知道“当前的累计时间”来计算当前飞机的延误成本。
   - **难点**：如果我们直接把“时间”作为状态的一部分，状态空间会太大。
   - **优化**：注意到总时间 = `（已起飞总数 - 1）+ 额外延误次数`。
     - 基础间隔是 1 分钟。
     - 只有当 `前一架 > 当前架` 时，间隔变为 2（即产生了 1 分钟的额外延误）。
     - 因此，我们可以引入一个状态维度 `inv` (inversion)，表示产生了多少次“额外延误”。
     - **当前时间** $T = (i + j + k - 1) + inv$ （针对第 $i + j + k$ 架飞机，如果是第1架则 $T = 0$）。
3. **DP 表设计：**
   - 状态：`dp[i][j][k][last][inv]`
     - `i, j, k`：分别表示 L、M、H 类飞机已经起飞的数量。
     - `last`：上一架飞机的类型（0:L, 1:M, 2:H, 3:Start）。
     - `inv`：累计的额外延误次数（即发生 $Last > Curr$ 的次数）。
   - 值：`min_cost`，达到该状态的最小总等待成本。
   - 同时需要记录 `parent` 表以便回溯输出路径。

算法复杂度

- 设 N_L, N_M, N_H 为各机型数量，N 为总数。
- 状态数：$N_L \times N_M \times N_H \times 3 \times N$。
- 约为 $O(N^4)$ 或 $O(N^5)$ 级别。对于 $N = 5$ 非常快；对于 $N = 50$ 左右也能在合理时间内跑完（python

序章：智能质变

东方盛夏 11月20日 20:54:13

Gemini 3 还是太全面了！
我之前有个想法，结合统计力学/热力学场景设计编程题（如系宗/铁磁相变/伊辛模型，有些现象非常妙趣），能兼具趣味/实用/深度，能激发学生探索挑战。
但是，直接按理论做计算，只是模拟仿真，不适合竞赛，最好能把相关物理问题转成典型算法题，要让学生去找出最优算法，而不是拿现成的物理公式去算。

东方盛夏 11月20日 20:54:21

我用了很多大模型去分析设计这种题，但它们都只停在原始物理理论上，没能转换出一个很巧妙的算法题。但今天用 gemini 3，再次一击成功，它的洞察和融通能力超强，在对各领域深刻理解基础上，准确将物理问题的某个面，和一些经典算法联系起来，输出了完美的竞赛题。

东方盛夏 11月20日 20:54:24

这里它将伊辛模型的基态能量求解，和竞赛顶流算法“最小割/网络流图”关联起来，它显著优于物理教材中使用的 metropolis 模拟法，冠军队伍可能会冒出这个天才想法，更多队伍会使用模拟法（近似解）或穷举法（指数级复杂度），得分梯度自然形成。
在对话引导下，gemini 还分析了几种解法的复杂度，并按实际物理效应，给出了测试用例生成程序，所有结论和程序都被验证正确，而且是一次成功。

User

以统计力学/热力学的核心理论/思想为基础（如伊辛模型/相变/系综等），设计几个有趣、实用、有深度、适合单人单机 2 小时竞赛的编程题，要便于输入输出和判题，给出题目描述、解析和题解程序

Model

Thoughts

Expand to view model thoughts

这是一份为时长 2 小时的单人编程挑战设计的题目集。这些题目将统计力学和热力学的核心概念（如能量极小化、配分函数、相变）映射到经典的算法模型（网络流、动态规划、并查集）上。

导言：编程与物理的边界

本次挑战的主题是**“计算宇宙的微观状态”**。你需要编写程序来模拟微观粒子的行为，计算系统的熵，或寻找系统的基态。

题目列表：

1. **题目 A：伊辛模型的基态 (The Ground State of Ising Model)** - *难度：Hard*
   - 核心思想：能量极小化、铁磁性
   - 算法：最小割/最大流 (Min-Cut/Max-Flow)
2. **题目 B：玻色气体的配分函数 (Partition Function of Bosons)** - *难度：Medium*
   - 核心思想：系综统计、能级简并
   - 算法：动态规划 (DP) / 完全背包变形
3. **题目 C：渗流阈值 (The Percolation Threshold)** - *难度：Easy*
   - 核心思想：相变、聚类
   - 算法：并查集 (Union-Find)

题目 A：伊辛模型的基态 (The Ground State of Ising Model)

1. 物理背景

伊辛模型（Ising Model）是统计力学中描述铁磁相变的最简化模型。系统由许多自旋（Spin）组成，每个自旋 s_i 只能取 $+1$（向上）或 -1（向下）。系统的总能量（哈密顿量）定义为：

$$H = -\sum_{\langle i,j \rangle} J_{ij} s_i s_j - \sum_i h_i s_i$$

其中：

- $J_{ij} > 0$ 表示相邻自旋 i, j 之间的铁磁耦合强度（倾向于同向）。
- h_i 是作用在自旋 i 上的外部磁场。

自然界总是倾向于演化到能量最低的状态（基态）。你的任务是确定所有自旋的取向，使得系统总能量 H 最小。

2. 题目描述

给定一个包含 N 个节点的无向图，节点代表自旋。
给定每条边 (u, v) 的耦合常数 J_{uv}（均为正数）。
给定每个节点 u 的外场强度 h_u（可正可负）。
请输出系统的最小能量值。

输入格式：

- 第一行两个整数 N, M，表示自旋数量和相互作用边的数量。
- 接下来 M 行，每行三个整数 u, v, w，表示 u 和 v 之间 $J_{uv} = w$。
- 最后一行 N 个整数，第 i 个数表示 h_i。

输出格式：

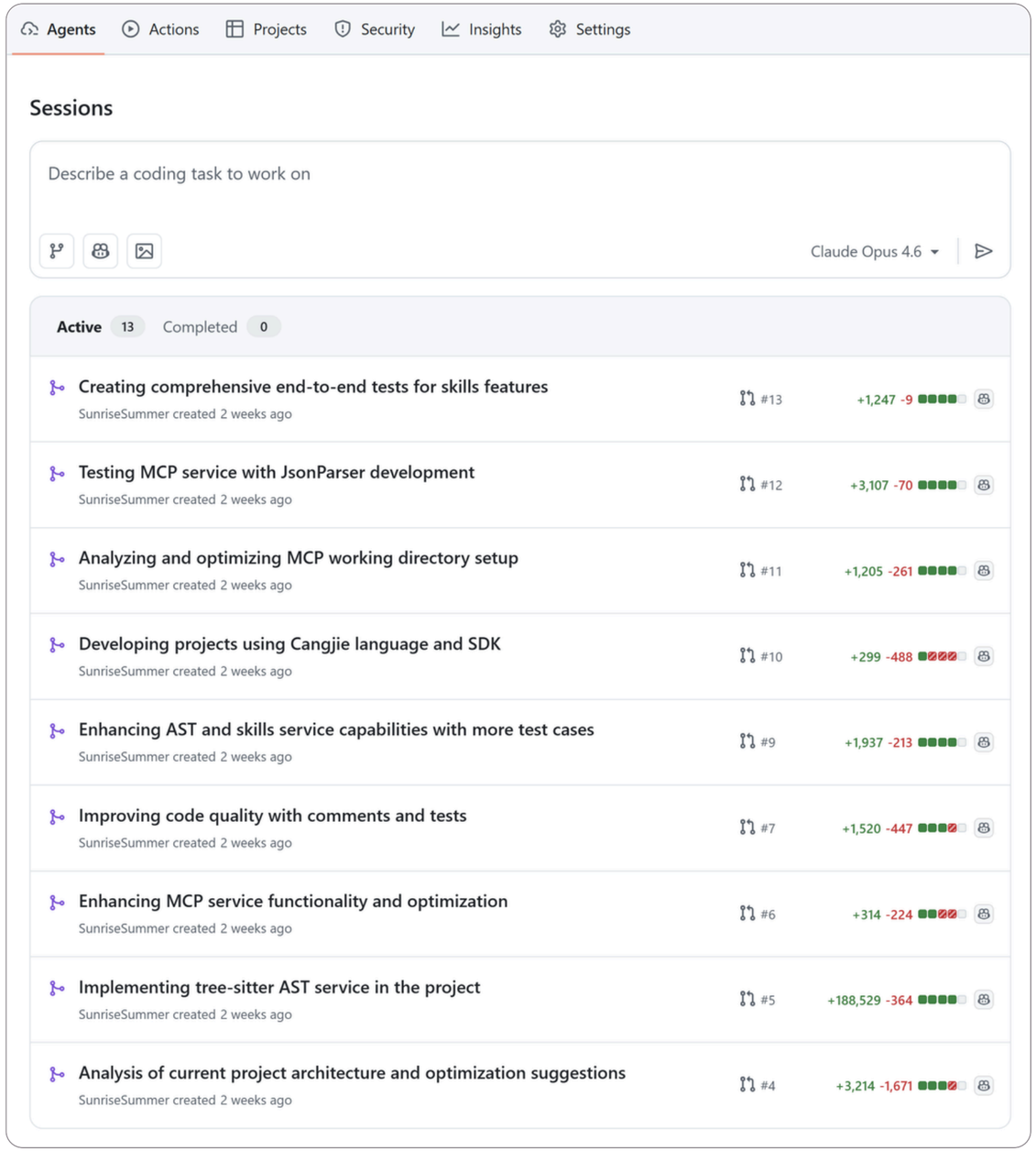
- 一个整数，表示最小总能量。

数据范围：

- $N \le 500, M \le 5000$
- $1 \le w \le 100$
- $-100 \le h_i \le 100$

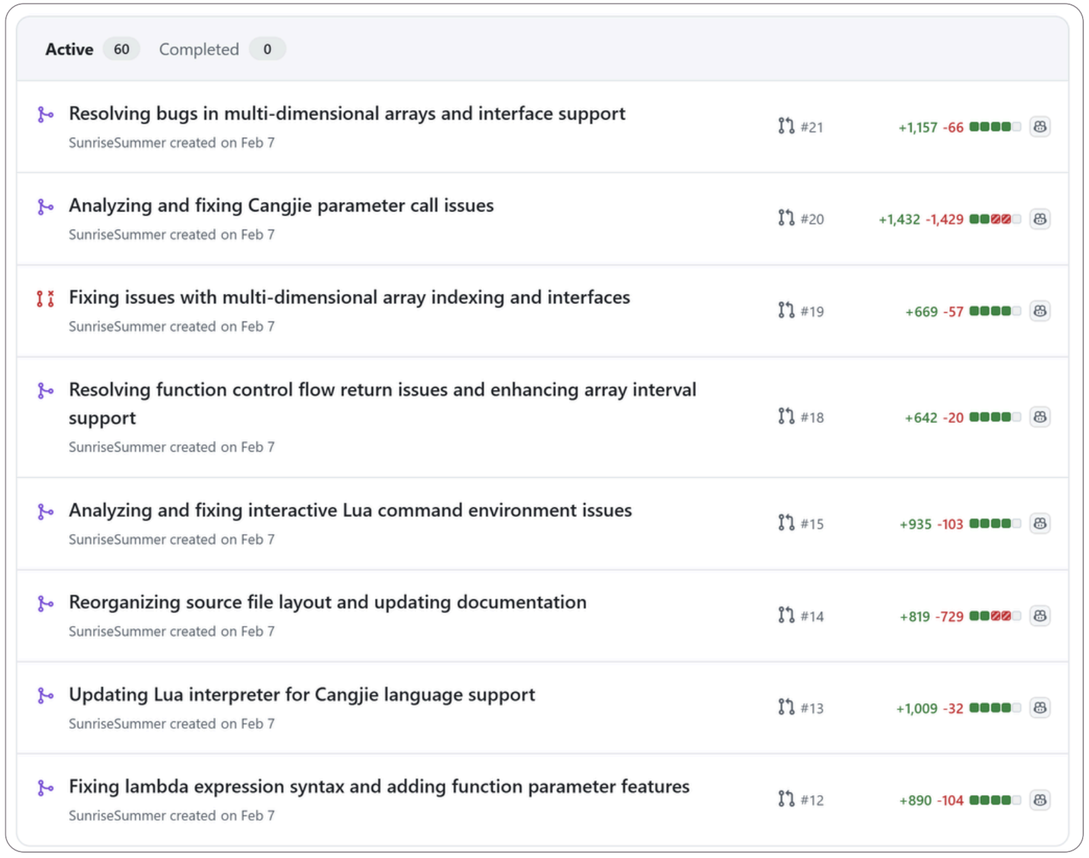
3. 解析与算法

核心转换： 这是一个典型的**最小割 (Min-Cut)** 问题。
我们想最小化 H。注意到 $s_i s_j$ 只有两种取值：同向为1，反向为-1。
我们可以重写能量公式。为了利用最小割模型（所有割边容量必须非负），我们需要将问题转化为“最小化损失”。

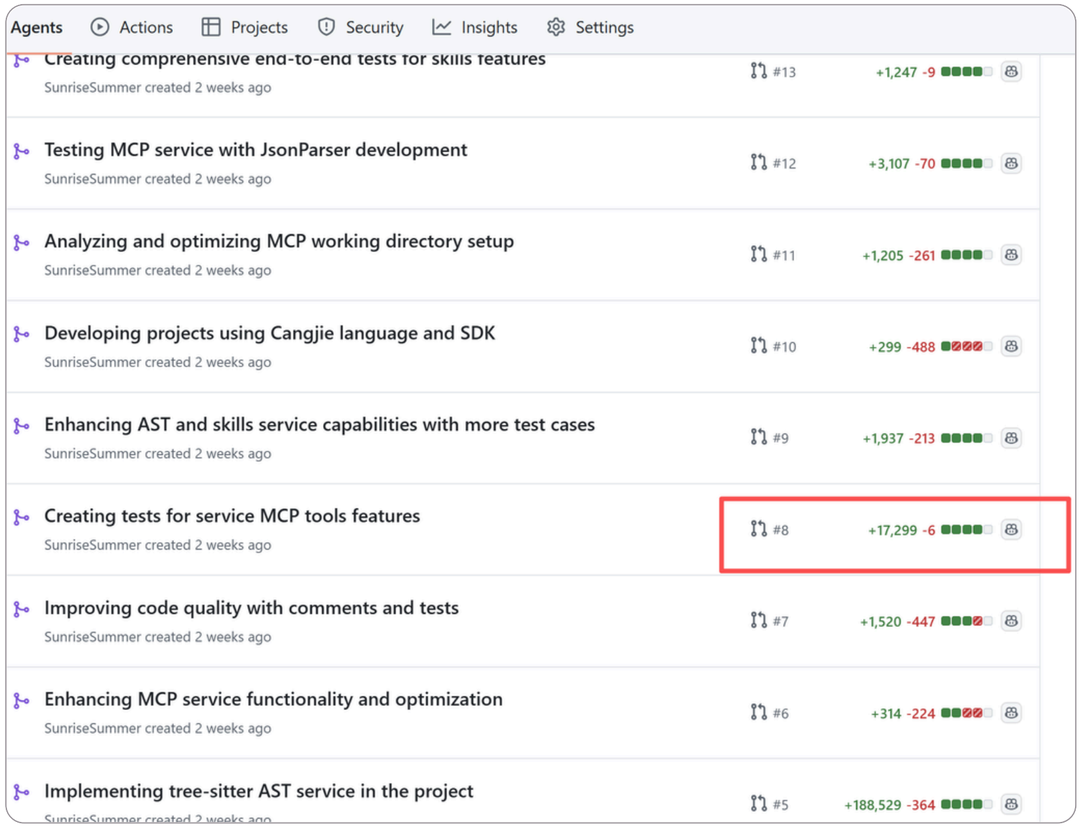


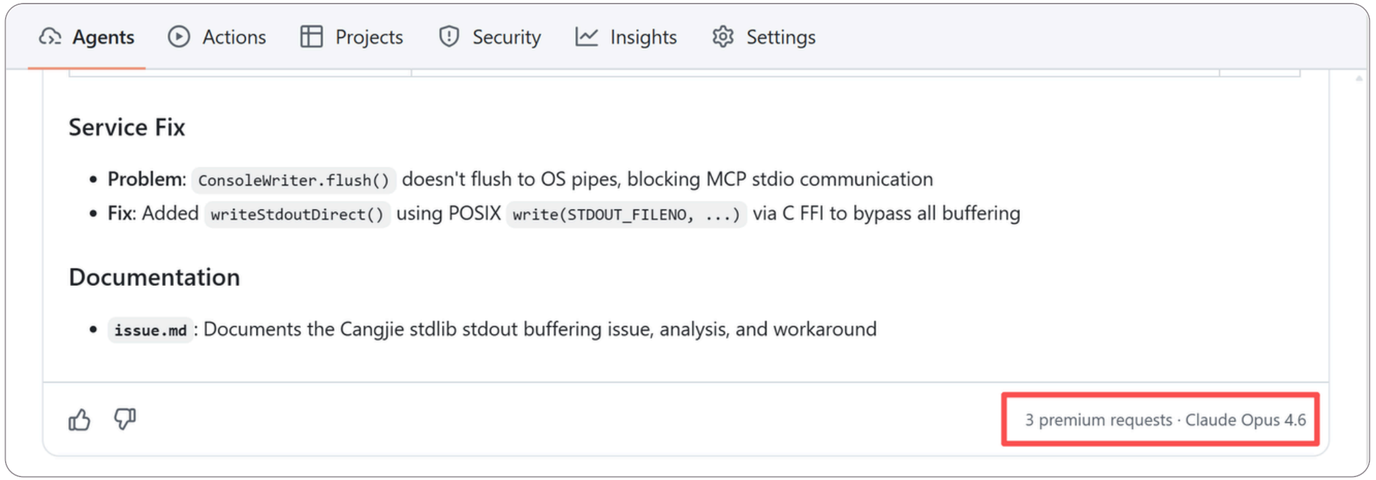
我常用的 AI 开发工具

1. github copilot agents 云开发
2. vscode + copilot agents 本地开发
3. opencode + minimax/kimi/glm 本地开发

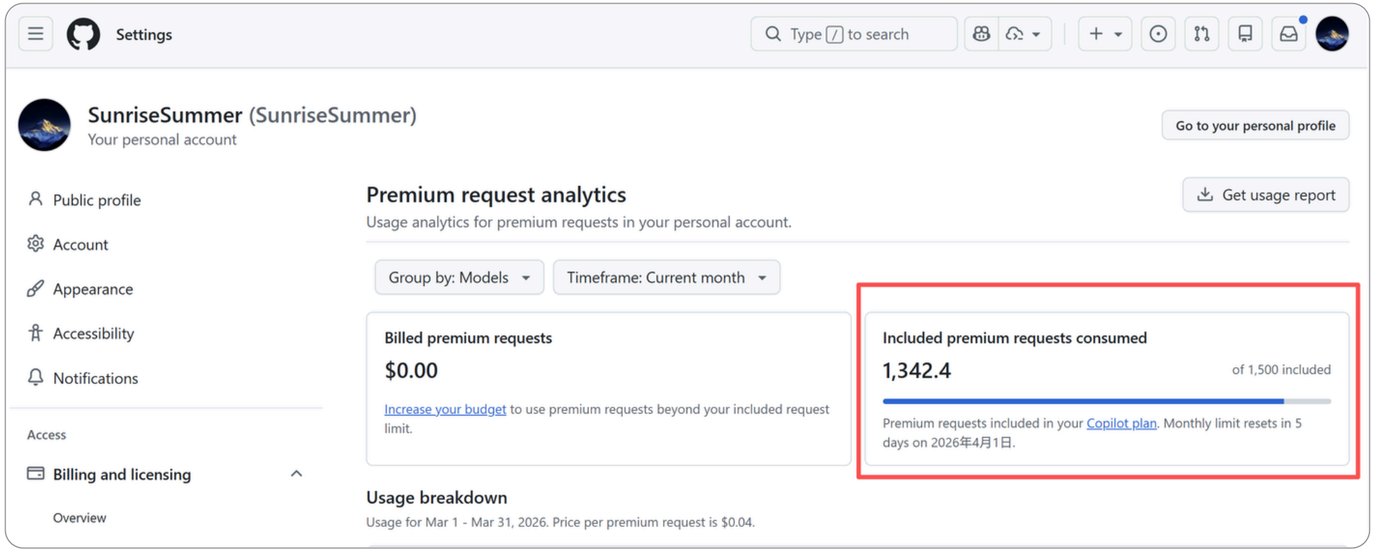


Github Copilot Agents：当前高性价比选项





按任务计费，在合理范围给任务上强度



高强度开发，配额充足

4. 架构图解

整个调用链路如下：

Code snippet

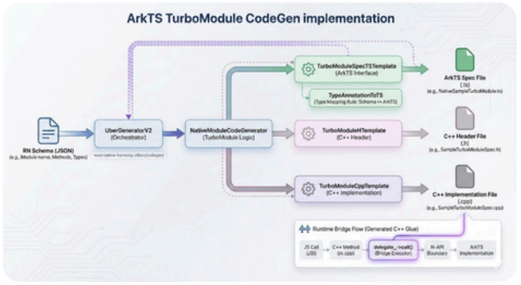
```
graph LR
    JS[JavaScript (RN)] -->|JSI Call| CPP_Glue[生成的 C++ TurboModule]
    CPP_Glue -->|delegate_->call()| ArkTS_Channel[ArkTSChannel (C++)]
    ArkTS_Channel -->|N-API (napi_call_function)| ArkTS_Impl[ArkTS TurboModule 实
```

总结

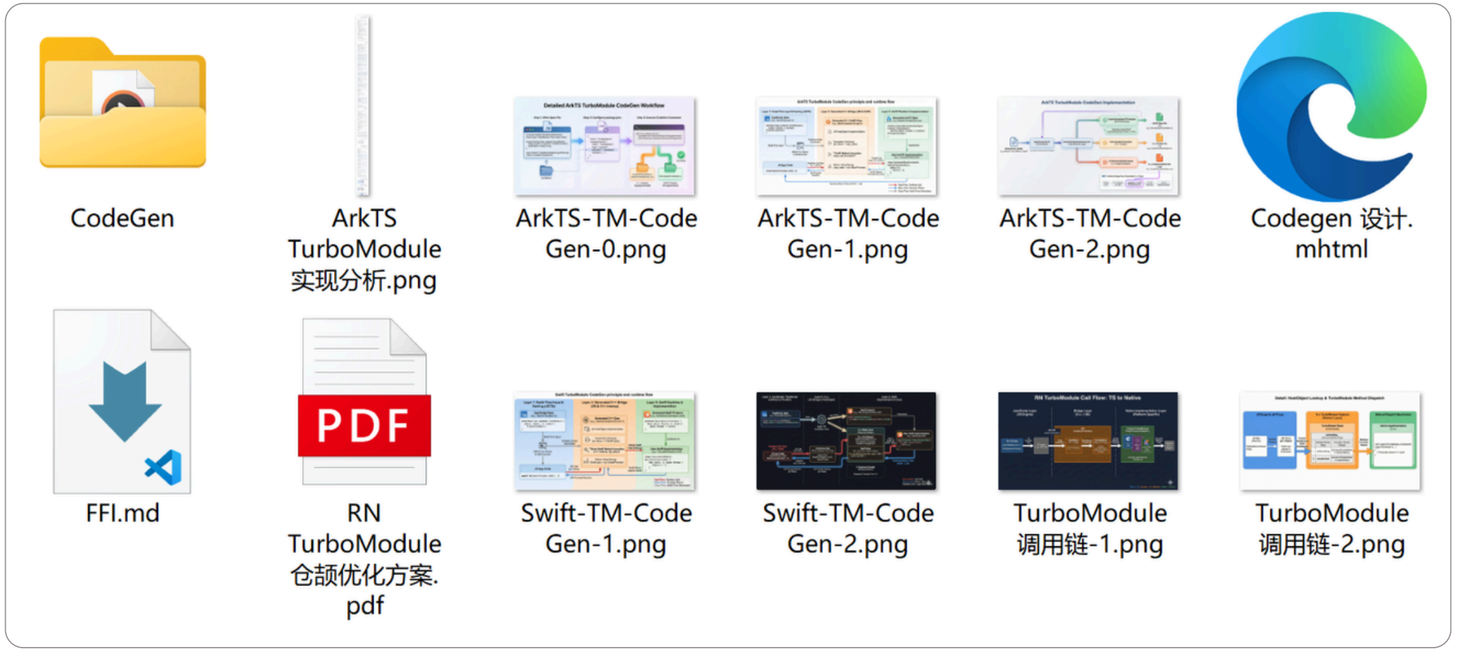
`delegate_` 实际上是一个**封装了 N-API 操作的执行器**。Code Generator 生成的代码并不关心具体的 N-API 细节（如 `napi_get_named_property`，`napi_create_string_utf8` 等），它只需要知道如何将 JSI 参数传递给 `delegate_`，剩下的跨语言互操作脏活累活全由 `delegate_` 指向的 `ArkTSChannel` 负责完成。这种设计极大地简化了代码生成的逻辑，并提高了代码的可维护性。

画图展示以上 CodeGen 实现细节，专业又生动，视觉效果优雅

Show thinking (Nano Banana Pro)



实战案例：RNOH TurboModule 适配仓颉语言



- Gemini 调研 + 方案设计
- Nano Banana 绘制架构图/方案图
- NotebookLM 制作汇报材料
- 开发：Copilot + Claude/GPT

仓颉 TurboModule Codegen 开发过程

Creating CodeGen for Cangjie TurboModule

Give feedback ··· Open in VS Code

You created the session last month · GPT-5.2-Codex · 3 sessions · 3 premium requests

You started a task

本仓库是适配鸿蒙系统的 React Native 框架（简称 RNOH）在 /tester/harmony/react_native_openharmony/src/main 目录下，我们使用仓颉编程语言实现了 RN 内置的一个 TurboModule——ImageLoader，之前是用 ArkTS 实现的，但仓颉的性能和互操作效率都比 ArkTS 好，所以我们改用仓颉实现后，测试发现性能有明显提升。 在那个目录下：

- cangjie 目录中是仓颉实现 ImageLoader 的核心逻辑
- cpp/RNOHCorePackage/TurboModules/ImageLoaderTurboModule.cpp 是原方案中 CodeGen 生成、作为桥接层调 ArkTS 模块的代理，现在被修改成调仓颉模块
- 由于 C++ 调仓颉需要封装一些类型转换、注册机制相关代码，我们把这些可复用代码放在 cpp/RNOHCangjieBridge 目录中，ImageLoaderTurboModule.cpp 中会引用。

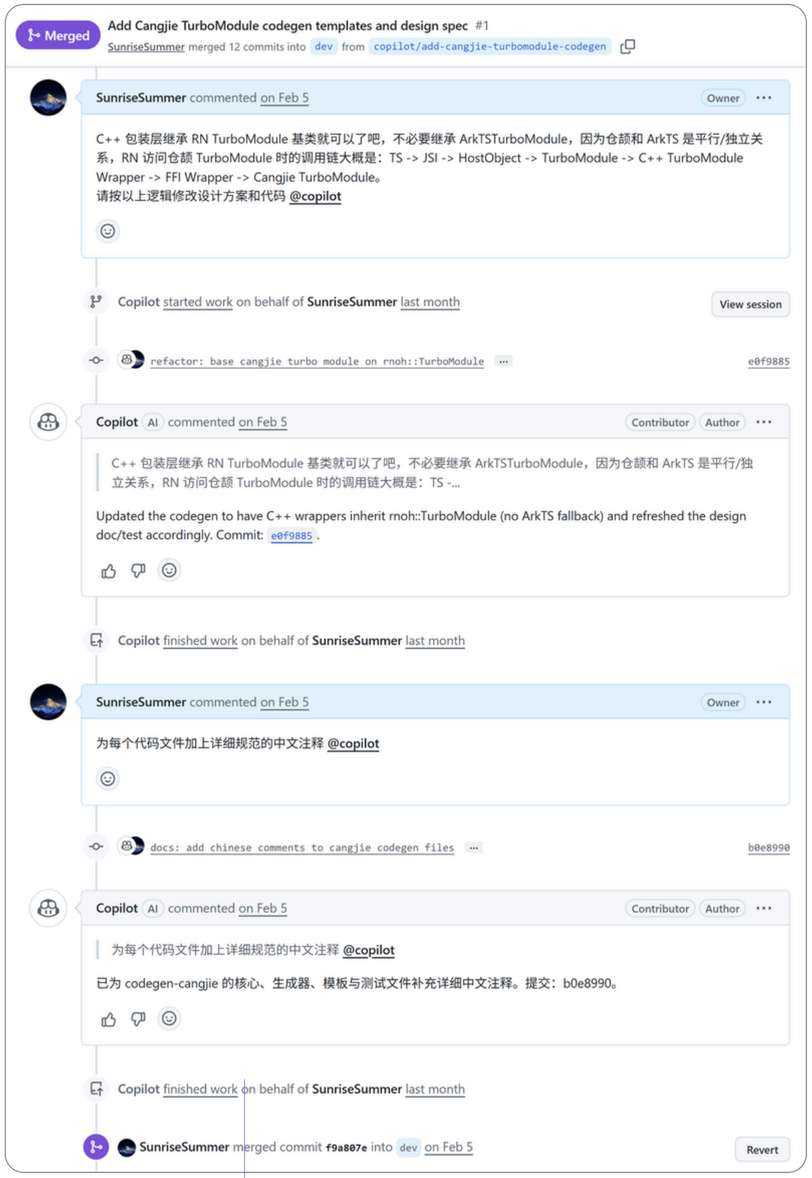
以上仓颉 TurboModule 是完全手工实现的，现在我们想为仓颉自定义 TurboModule 实现一个 CodeGen，可以生成 TurboModule 模板，复用更多代码，提升开发效率。请你根据当前已经实现的 ImageLoader 代码，并结合 RN TurboModule Codegen 工具的实现机制（可参考 react-native-harmony-cli/src/codegen，或 RN 为其他语言实现的 CodeGen），为仓颉 TurboModule 模板 Codegen 设计一套具体方案，输出详细设计的 markdown 文件（在项目根目录），并尝试在 react-native-harmony-cli/src/codegen-cangjie 目录下实现这个方案，但不必测试（因为你没有相关构建运行环境），我会人工检视和测试。

Show less

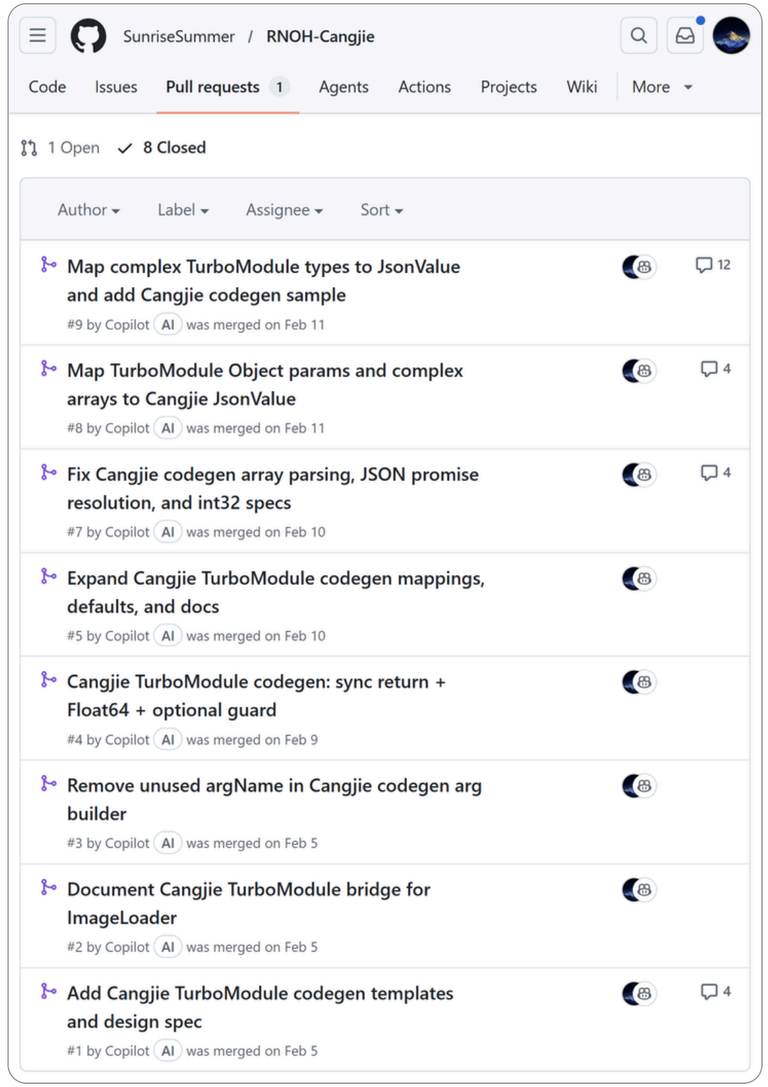
Copilot started work

提需求

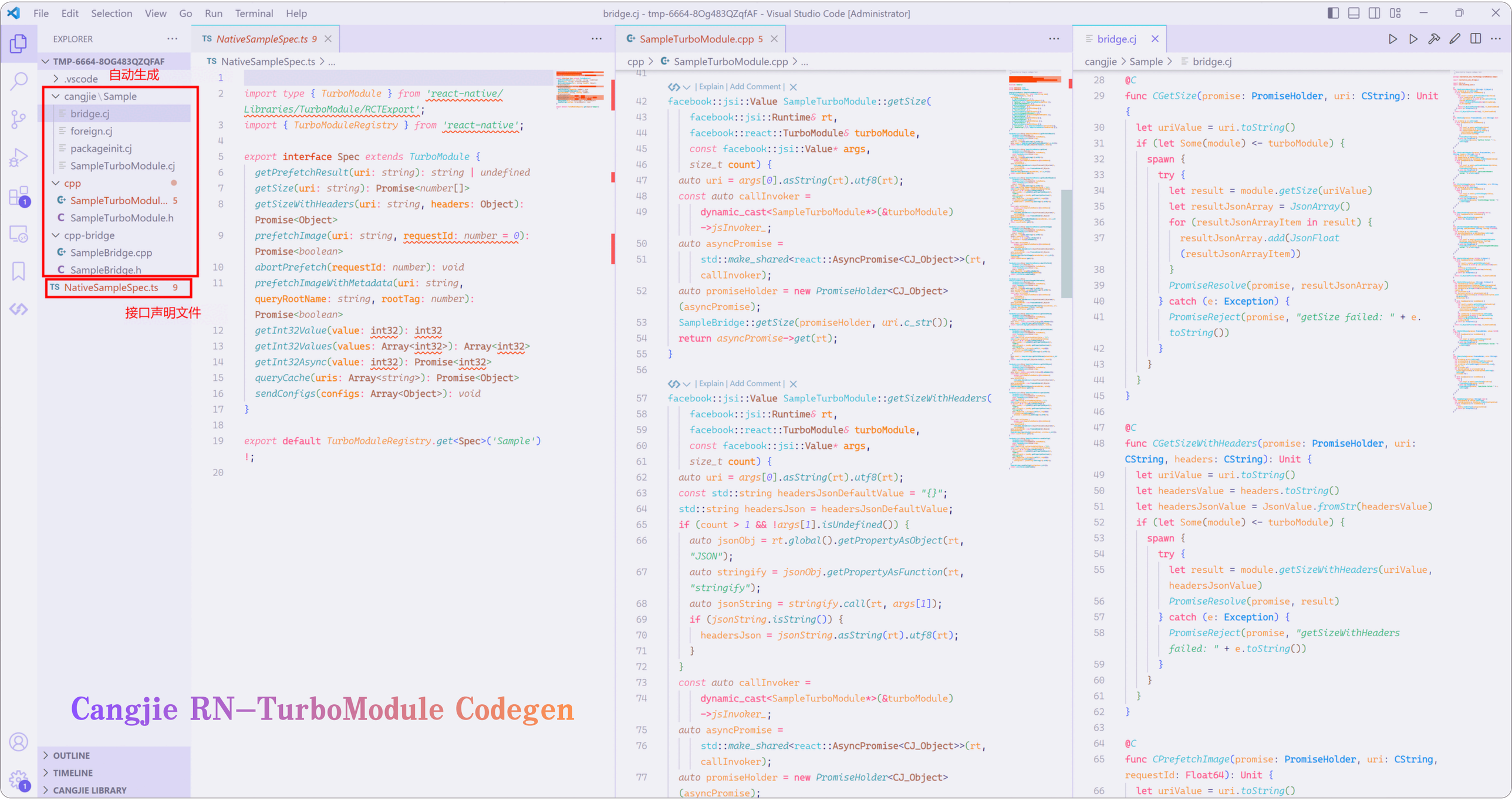
Creating CodeGen for Cangjie TurboModule ✓ 37m 3s ···



检视



迭代


File Edit Selection View Go Run Terminal Help
bridge.cj - tmp-6664-8Og483QZqfAF - Visual Studio Code [Administrator]
EXPLORER
TMP-6664-8OG483QZQFAF
.vscode
自动生成
cangjie \ Sample
bridge.cj
foreign.cj
packageinit.cj
SampleTurboModule.cj
cpp
SampleTurboModul... 5
SampleTurboModule.h
cpp-bridge
SampleBridge.cpp
SampleBridge.h
NativeSampleSpec.ts 9
接口声明文件
OUTLINE
TIMELINE
CANGJIE LIBRARY
NativeSampleSpec.ts 9
SampleTurboModule.cpp 5
bridge.cj
cpp > SampleTurboModule.cpp > ...
cangjie > Sample > bridge.cj
Cangjie RN–TurboModule Codegen

AI 开发经验：品味驱动，抓大放小，测试看护

- AI 交的作业错误很少，但技术品味和用户体验需要我们把关。
- AI 开发亩产万斤，传统检视方法已经失效，我们要像领导一样抓大放小，更关注架构/规范/美学。
- 引导 AI 构造巧妙的、充分的测试用例，以此看护我们放过的代码细节、保证产品质量。

You started a task

当前从复杂 .mid 解码出的乐谱语言，和 testcase/source 中的友好形式差异较大（可读性较差），更大的问题时，把它们再编译成 .mid 文件后，播放起来失真严重，我认为当前的 MIDI 语言设计有问题，一方面不够专业和友好（写乐谱不够顺手，尤其是涉及和弦、多乐器、多音轨等复杂编曲时），另一方面又束缚了反编译 .mid 时的表达能力，导致不能无损还原。请重新设计这个语言，既专业友好易用，同时还能无损转译任何 .mid 音频。

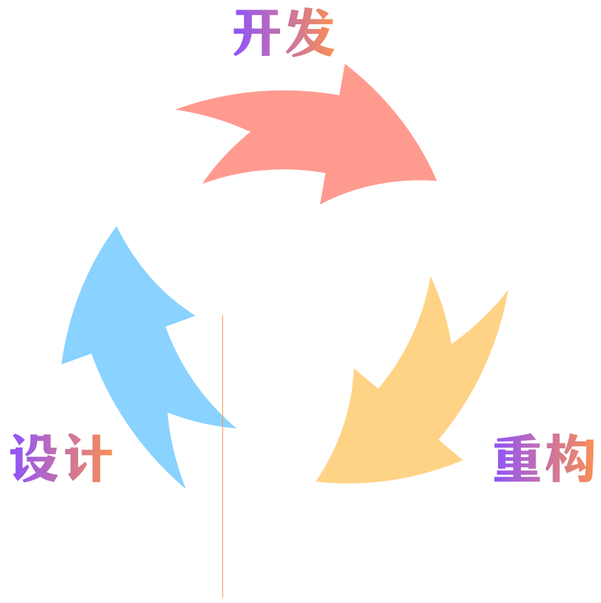
- 移除当前项目中的两种语言设计（乐谱记谱和可读事件格式），设计实现完美的新语言
- 新语言要借用现代高级编程语言的一些设计思想，让写乐谱/编曲像编程一样灵活强大，我相信这种设计能轻松面对复杂编曲/音效场景，当前乐谱语言在编排时值/音高/节拍/休止/和弦等状态时有很多冗余符号和重复模式，完全可以用程序结构高效抽象表达。
- 基于 testcase/binary 下的 .mid 文件测试无损编解码。
- 在 testcase/source 下补充更多复杂乐谱，充分测试。

改造后充分测试验证，并更新相关文档。

Refactor agent context management, JSON parsing, LSP persistent connection, rollback, session persistence, and sub-package splitting #4

Merged SunriseSummer merged 17 commits into main from copilot/analyze-architecture-issues 3 weeks ago

Conversation 18 Commits 17 Checks 0 Files changed 42 +3,214 -1,671



SunriseSummer commented 3 weeks ago Owner

@copilot 检查和优化项目中的异常处理机制，有两个原则导向：

- 不造成阻断性影响的问题，尽量有逃生通道/兜底机制，而不要敏感频繁地终止服务，要提升服务鲁棒性
- 优先用 Option 表达和处理异常，如有必要才使用 throw-try-catch 机制

降低圈复杂度

- `analyzer.cj`：将 `analyzeCangjieFile()` 拆分为 `tryTreeSitterAnalysis` / `tryCjlintAnalysis` / `tryCjcAnalysis` 三个子函数，通过 `Option` 链式调用串联：

```
let treeSitterResult = tryTreeSitterAnalysis(fullPath, repoRoot)
if (let Some(result) <- treeSitterResult) { return result }
let lintResult = tryCjlintAnalysis(fullPath, repoRoot)
if (let Some(result) <- lintResult) { return result }
return tryCjcAnalysis(fullPath, repoRoot)
```

- `protocol.cj`：`buildToolDefinitions()` 从 70+ 行单一数组字面量改为 `ArrayList` 逐步构建，按工具类别（技能/文件/构建/分析/LSP/模板/回滚）分组
- `helpers.cj`：`parseJsonBoolField` 移除冗余空值前置判断
- `protocol.cj`：`mcpToolCallResultJson` 中 `isError` 判定从嵌套 match 简化为 `value.toString() != "true"`

重构异常处理机制

按两个原则重构：(1) 不造成阻断性影响的问题提供兜底机制，不终止服务；(2) 优先用 Option 表达异常，解构时用 if-let 而非 match case。

消除全部 13 处 throw 语句：

函数	原始行为	优化后
`parseNodeCoordinate`	throw 坐标解析失败	返回 `?((Int64,Int64),Int64)`
`offsetForPoint`	throw 坐标越界	返回 `?Int64`
`ensureWorkspacePath`（4 处副本）	throw 路径不存在/逃逸	返回 `?String`
`ensurePlannedWorkspacePath`（3 处 throw）	throw 父目录/逃逸	返回 `?String`
`LspSessionManager.sendFrame`	throw 进程未启动	返回 `Bool`

统一 if-let 解构，包括多条件 `&&` 组合、嵌套 if-let、`??` 兜底等模式。

保留 try-catch 仅用于顶层安全网： `callMcpTool`、`McpRuntime.handleRequest`、外部命令执行、LSP 子进程通信等不可避免的 I/O 异常场景。

补充中文注释

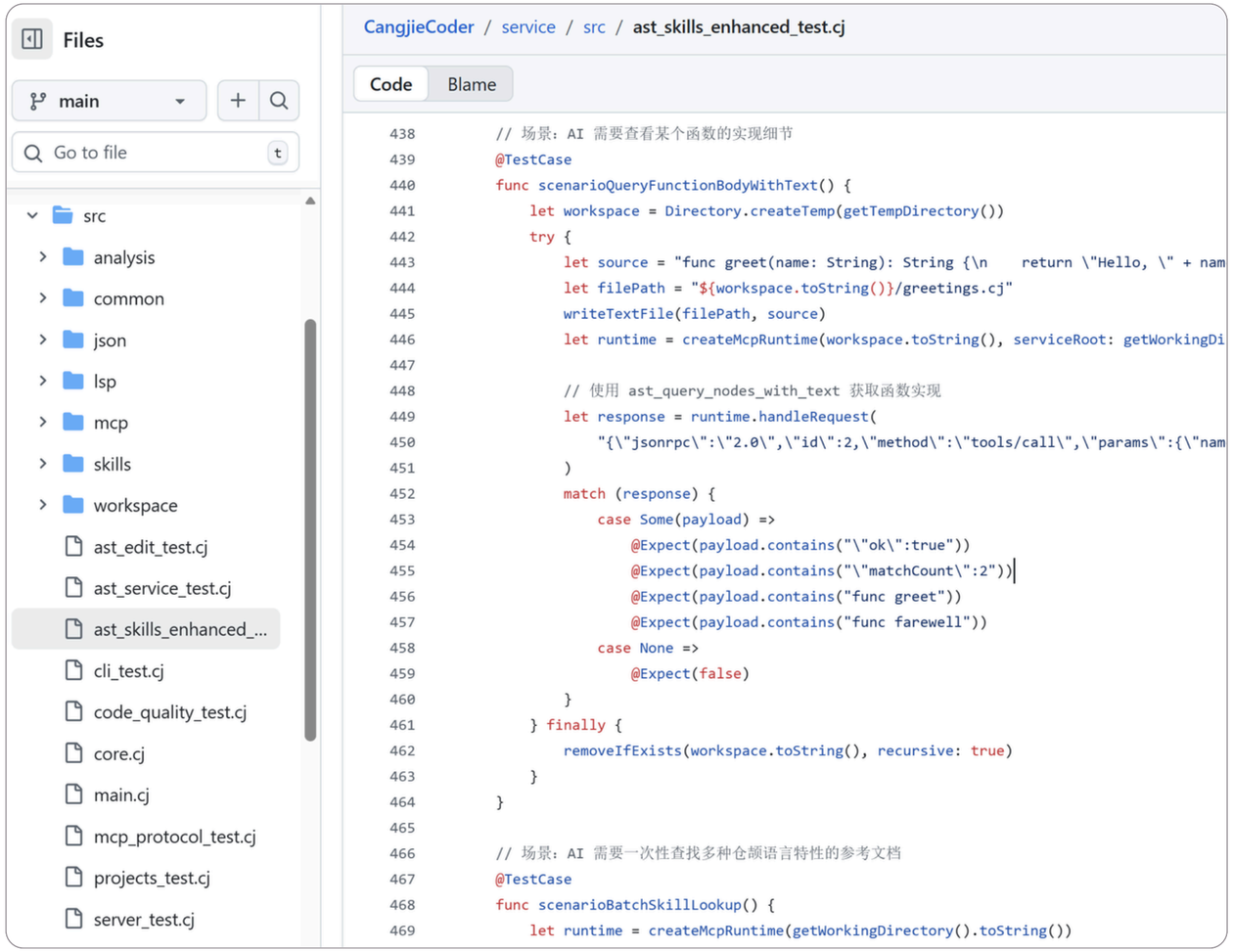
所有 7 个文件（含 `lsp/protocol.cj`）添加：

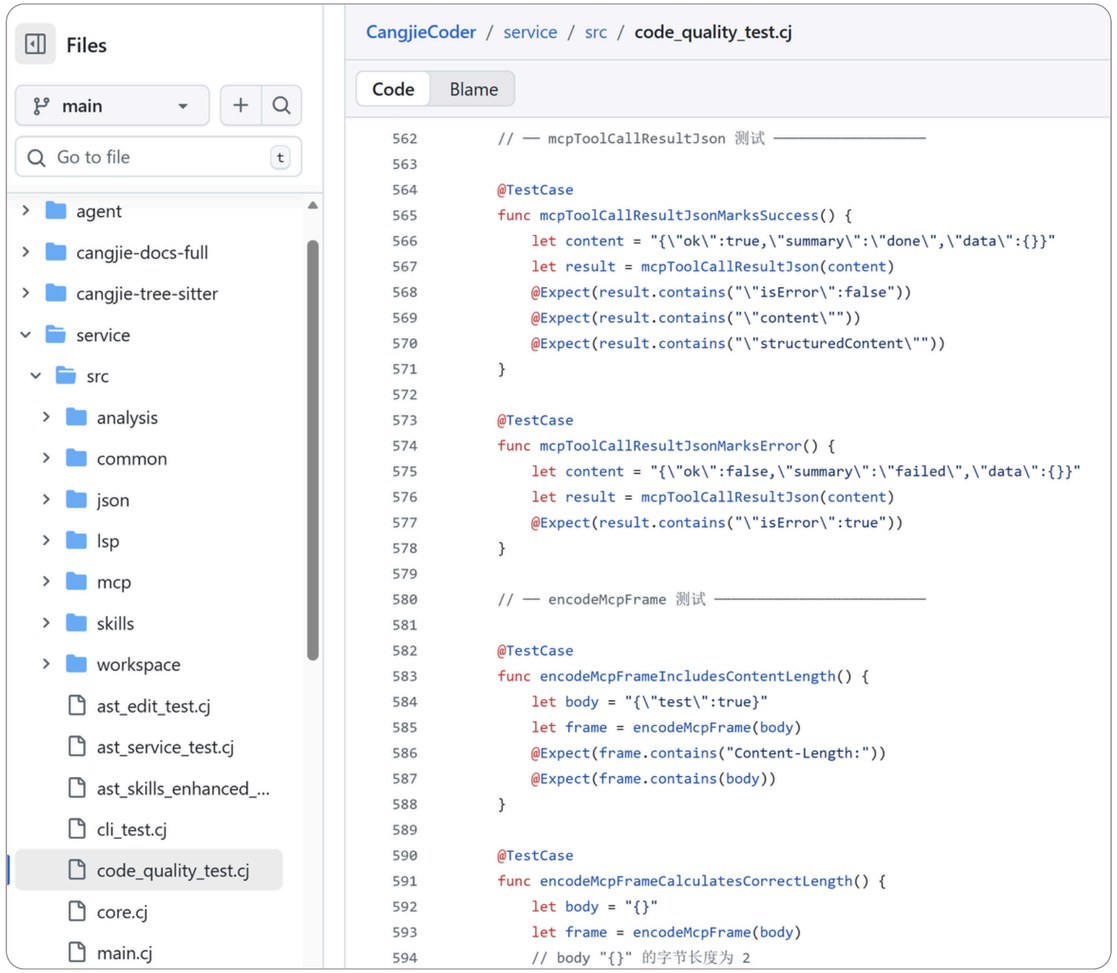
- 文件级模块职责描述
- `// — 段落名 —` 风格代码分段标记
- 函数级用途/参数/返回值/边界条件文档
- 关键逻辑行内注释（协议交互、状态机转换等）
- 统一 `catch (_: Exception)` 命名规范

新增测试

`service/src/code_quality_test.cj` — 5 个测试类，64 个用例：

充足的单元测试看护基本盘





复杂场景要巧妙构造端到端测试

You started a task

现在 service 提供的 MCP 工具已经初步完善，想请你做一个测试：

构建并启动 MCP 服务（仓颉 SDK：https://gitcode.com/Cangjie-SIG/CangjieSkills/releases/1.0.5.1 ）

你本身就是一个通用 Agent，因此希望你模拟基于这些 MCP 工具的仓颉项目 AI 开发过程，例如开发一个 JsonParser 项目（覆盖标准语法），过程中你尽可能只调用当前 MCP 服务，完成项目开发，整个过程都用 python 脚本记录下来（其中有些代码，可能是你思考后直接输出的内容，其他内容就是显式调用 MCP 工具在分析和修改项目），最终保证运行这个 python 脚本（保存到 tests/e2etest_jsonparser.py），能重放整个开发过程，在测试目录生成可用的 JsonParser 项目，实现对 service 真正的端到端测试。

端到端测试发现模块组合使用时的问题

```
+        # ============================================================
+        # 阶段 3 — AST 深度分析：全面使用 AST 工具链
+        # summary / parse / list_nodes / query_nodes / query_with_text
+        # ============================================================
+        step("阶段 3: AST 深度分析 — 多工具协同审查代码结构")
+
+        agent_plan("AST 多维度分析", [
+            "ast_summary: 获取每个文件的顶层定义摘要（快速概览）",
+            "ast_parse: 获取 main.cj 的完整 S-expression AST",
+            "ast_list_nodes: 列出 task.cj 前 3 层的所有命名节点",
+            "ast_query_nodes: 统计 types.cj 中的枚举定义数量",
+            "ast_query_nodes_with_text: 提取 task.cj 中所有函数的源码",
+        ])
+
+        c = client()
+        c.start()
+        c.call_tool("workspace.set_root", {"path": workspace})
+        # ast_summary — 各文件顶层定义
+        c.call_tool("cangjie.ast_summary", {"path": "src/types.cj"})
+        c.call_tool("cangjie.ast_summary", {"path": "src/task.cj"})
+        # ast_parse — 完整 AST
+        c.call_tool("cangjie.ast_parse", {"path": "src/main.cj"})
+        # ast_list_nodes — 列出节点树
+        c.call_tool("cangjie.ast_list_nodes", {
+            "path": "src/task.cj", "maxDepth": 3
+        })
+        # ast_query_nodes — 统计枚举定义
+        c.call_tool("cangjie.ast_query_nodes", {
+            "path": "src/types.cj", "nodeType": "enumDefinition"
```

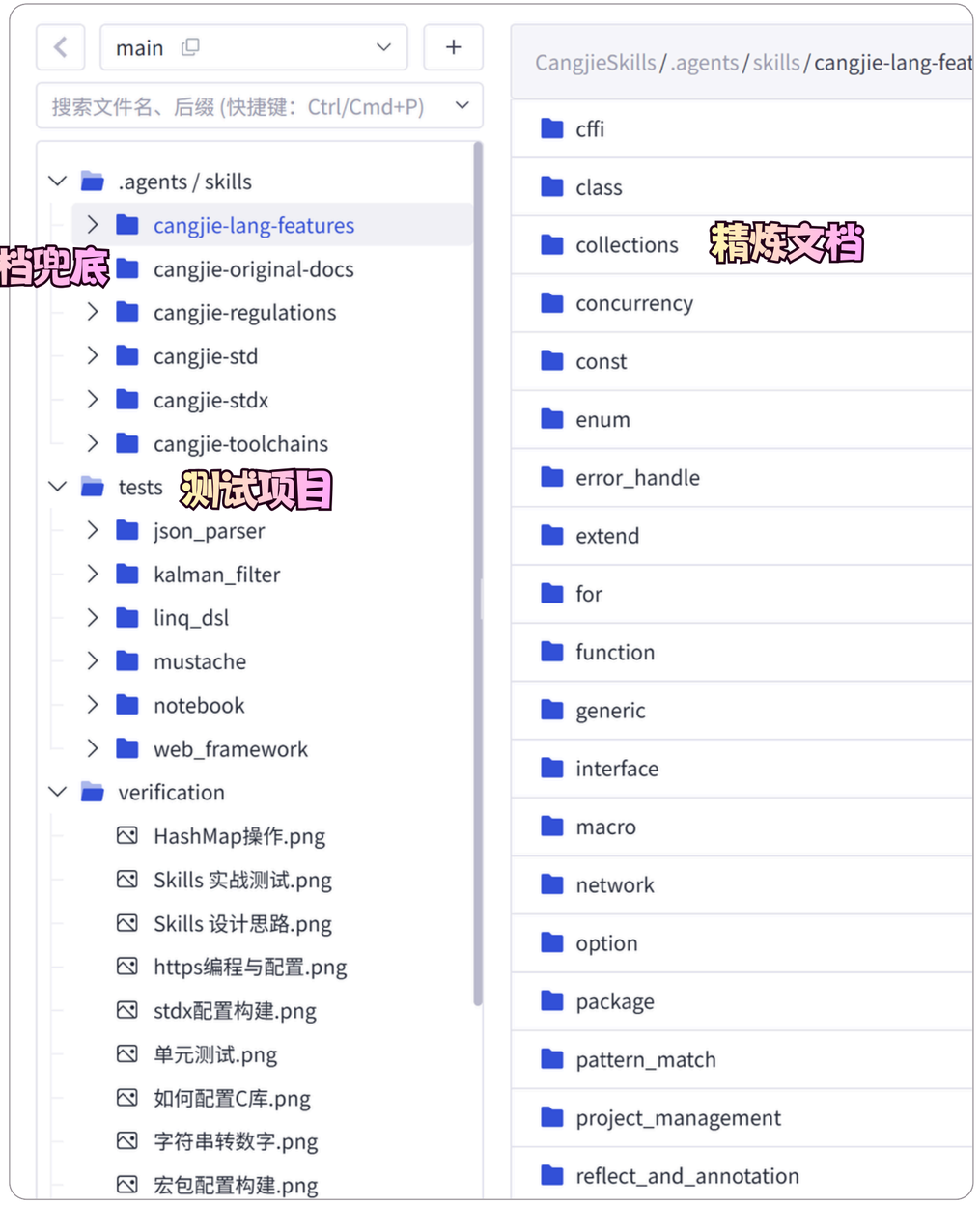
```
+        # ============================================================
+        # 阶段 4 — AST 编辑：edit_ast_node 替换函数 → 验证 → 回滚
+        # 这是本测试的核心亮点：演示 AST 级别的精确代码编辑
+        # ============================================================
+        step("阶段 4: AST 编辑 — 精确替换函数定义并回滚")
+
+        agent_plan("AST 编辑实验", [
+            "用 ast_query_nodes_with_text 找到 summary() 函数",
+            "用 edit_ast_node 替换 summary() 为增强版（含标签显示）",
+            "读取文件验证替换结果",
+            "用 workspace.rollback 恢复原始版本",
+            "再次读取文件确认恢复成功",
+        ])
+
+        # --- 4a: 先找到 summary() 是第几个函数 ---
+        c = client()
+        c.start()
+        c.call_tool("workspace.set_root", {"path": workspace})
+        c.call_tool("cangjie.ast_query_nodes_with_text", {
+            "path": "src/task.cj", "nodeType": "functionDefinition"
+        })
+        resp = c.execute()
+
```

Agents Actions Projects Security Insights Settings

```
阶段 4: AST 编辑 — 精确替换函数定义并回滚

Agent 规划: AST 编辑实验
  • 用 ast_query_nodes_with_text 找到 summary() 函数
  • 用 edit_ast_node 替换 summary() 为增强版（含标签显示）
  • 读取文件验证替换结果
  • 用 workspace.rollback 恢复原始版本
  • 再次读取文件确认恢复成功
✓ 找到 summary() 函数 (索引=3)
Agent 分析: summary() 是第 3 个函数定义节点。
AI 模型响应 ← codepiece7.ai (415 字节)
  提示词: 生成 summary() 的增强版: 在摘要中包含标签列表
✗ edit_ast_node: 替换函数定义成功
✗ 验证 AST 编辑: 新代码包含标签处理逻辑
Agent 分析: AST 编辑成功: summary() 已被替换为含标签显示的增强版本。
Agent 决策: 实验完成，回滚 AST 编辑以恢复原始代码。
✓ workspace.rollback: 回滚成功
✓ 验证回滚: 文件内容与原始一致
Agent 分析: 回滚成功: task.cj 已恢复到 AST 编辑前的状态。
  edit_ast_node + rollback 形成了安全的「实验→撤销」工作流。

阶段 5: 读取文件并验证内容

Agent 规划: 内容交叉验证
  • 读取 cjpm.toml 确认项目名
  • 读取 types.cj 确认枚举定义
  • 全局搜索 TaskStore 确认引用
✓ cjpm.toml 包含正确的包名
✓ types.cj 包含 Priority 和 TaskStatus 枚举
✓ search_text(TaskStore): 2 处引用
Agent 分析: 内容验证通过。准备首次编译。

阶段 6: 编译项目 — 迭代修复 import 错误

Agent 规划: 首次编译尝试
  • 执行 workspace.run_build
  • 预期: 编译失败（缺少 import std.convert.*）
  • 修复后重新编译
✓ 首次编译失败（预期: 缺少 import std.convert.*）
Agent 分析: 编译失败 (exit=1)。
  错误指向 Int64.parse() 调用 — 缺少 import std.convert.*。
  错误摘要: error: undeclared identifier 'parse'
==> /tmp/cjcoder_e2e_taskmanager_ufvd9t22/src/main.cj:38:24:
   |
38 |     let parsed = Int64.parse(numStr)

Agent 决策: 请求 AI 重新生成 main.cj，补充 import std.convert.*。
AI 模型响应 ← codepiece5.ai (1019 字节)
  提示词: 修复 main.cj: 添加 import std.convert.* 以支持 Int64.parse()
✓ create_file(main.cj, overwrite) 覆盖写入修复版

Agent 规划: 重新编译
  • import 修复后再次编译
✓ 重新编译成功 ✅
Agent 分析: 编译通过！准备添加测试。

阶段 7: 生成测试 — 迭代修复 @Derive 注解
```

端到端测试发现问题

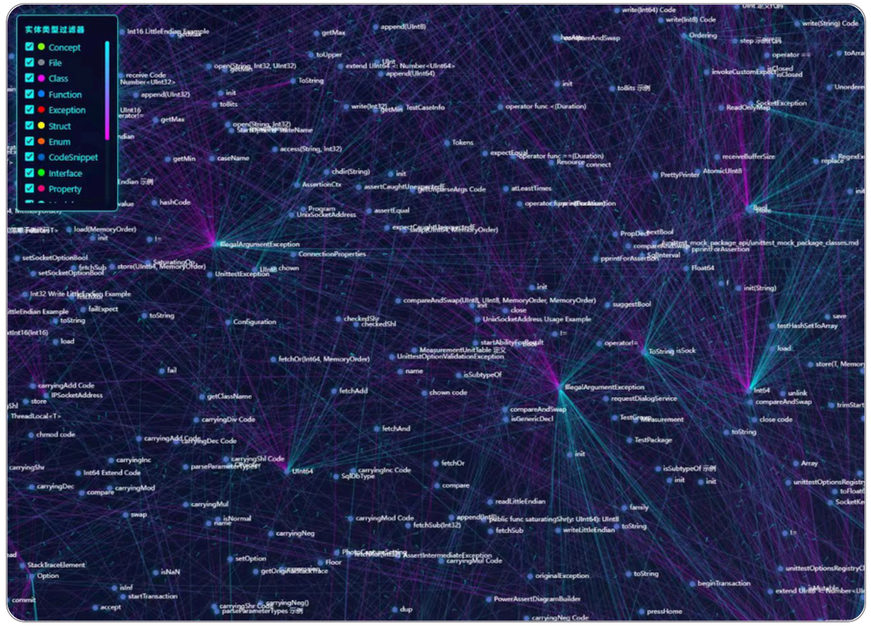
Cangjie Skills

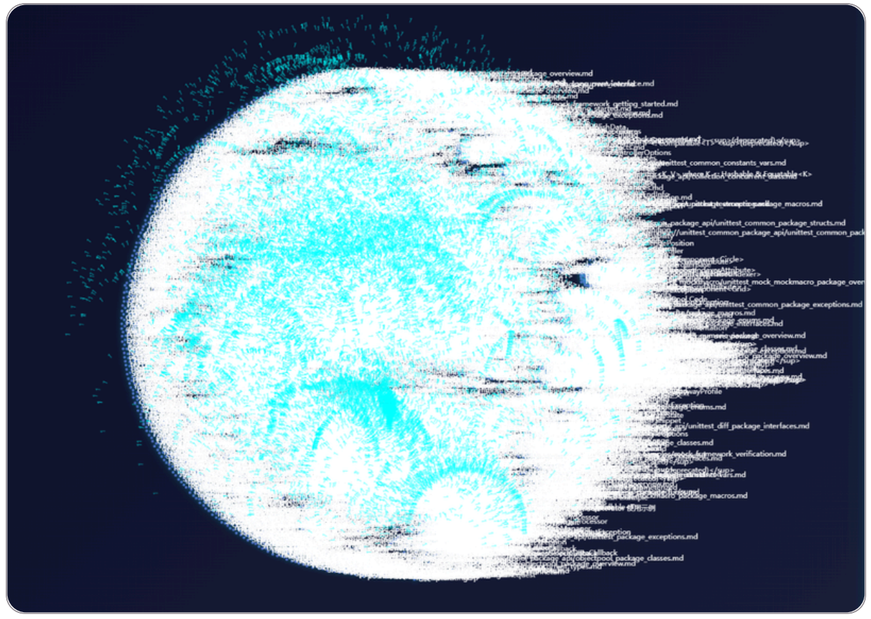


当前多数模型对仓颉训练支持较弱，仓颉相关知识库要为特性**提供完整上下文，避免知识碎片**。为此采用如下策略：

- 为语言核心特性编写精炼文档，并按加载使用的生命周期做拆分（如宏和宏构建拆开），SKILL.md 提供索引大纲。
- 扩展知识如鸿蒙API、三方库等，基于 OpenViking 提供文档检索，渐进式披露。SKILL.md 提供知识库使用指导。

知识库方案探索演进：结构化文档–向量混合搜索–知识图谱–OpenViking





知识图谱：精准灵活但成本高

OpenViking：高效准确但大颗粒

https://gitcode.com/Cangjie-SIG/CangjieOV

Github Copilot 如何构建测试仓颉项目

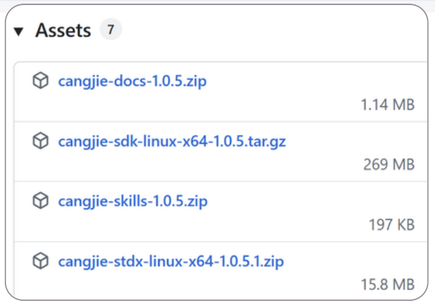
You started a task

当前从复杂 .mid 解码出的乐谱语言，和 testcase/source 中的友好形式差异较大（可读性较差），更大的问题时，把它们再编译成 .mid 文件后，播放起来失真严重，我认为当前的 MIDI 语言设计有问题，一方面不够专业和友好（写乐谱不够顺手，尤其是涉及和弦、多乐器、多音轨等复杂编曲时），另一方面又束缚了反编译 .mid 时的表达能力，导致不能无损还原。请重新设计这个语言，既专业友好易用，同时还能无损转译任何 .mid 音频。

- 移除当前项目中的两种语言设计（乐谱记谱和可读事件格式），设计实现完美的新语言
- 新语言要借用现代高级编程语言的一些设计思想，让写乐谱/编曲像编程一样灵活强大，我相信这种设计能轻松面对复杂编曲/音效场景，当前乐谱语言在编排时值/音高/节拍/休止/和弦等状态时有很多冗余符号和重复模式，完全可以用程序结构高效抽象表达。
- 基于 testcase/binary 下的 .mid 文件测试无损编解码。
- 在 testcase/source 下补充更多复杂乐谱，充分测试。

改造后充分测试验证，并更新相关文档。

仓颉 SDK 下载：
https://github.com/SunriseSummer/CangjieSDK/releases/download/1.0.5/cangjie-sdk-linux-x64-1.0.5.tar.gz



Build to verify zero warnings

```
$ cd /home/runner/work/CangjieLua/CangjieLua && source cangjie/envsetup.sh && cd midi && cjpm build 2>&1
cjpm build success
<exited with exit code 0>
```

Zero warnings! Now run the tests:

Run all tests

```
$ cd /home/runner/work/CangjieLua/CangjieLua && source cangjie/envsetup.sh && cd midi && cjpm test 2>&1
-----------------------------------------------------------------------------------------------
TP: midi, time elapsed: 120269359 ns, RESULT:
    TCS: MidiTests, time elapsed: 120269359 ns, RESULT:
    [ PASSED ] CASE: twinkleVariation (1321267 ns)
    [ PASSED ] CASE: decompileRoundTrip (704175 ns)
    [ PASSED ] CASE: velocityControl (191357 ns)
    [ PASSED ] CASE: extendedInstruments (297065 ns)
    [ PASSED ] CASE: gmInstrumentMapping (51567 ns)
    [ PASSED ] CASE: decompileExternalTest1 (25226376 ns)
    [ PASSED ] CASE: decompileExternalTest2 (22573764 ns)
    [ PASSED ] CASE: decompileExternalTest3 (44141892 ns)
    [ PASSED ] CASE: dottedNoteDurations (220141 ns)
    [ PASSED ] CASE: rawTickDurations (173034 ns)
    [ PASSED ] CASE: channelPreservation (226052 ns)
    [ PASSED ] CASE: tempoAtAbsolutePositioning (254756 ns)
    [ PASSED ] CASE: tempoRoundingPrecision (21644570 ns)
Summary: TOTAL: 13
    PASSED: 13, SKIPPED: 0, ERROR: 0
    FAILED: 0
-----------------------------------------------------------------------------------------------
Project tests finished, time elapsed: 128132840 ns, RESULT:
TP: midi.*, time elapsed: 128060357 ns, RESULT:
    PASSED:
    TP: midi, time elapsed: 120269359 ns
Summary: TOTAL: 13
```

把需要传入沙箱的文件放到某个 github public 仓库 release 板块
在 copilot 布置任务时给出下载地址（外部地址多被屏蔽）

实现仓颉编码自由

You started a task

当前项目是用仓颉编程语言实现的 Protobuf 库，现在请你新增一个 examples/js 目录，在其中基于 protobuf 能力实现仓颉和 node.js 的互操作：

- 仓颉程序作为宿主首先启动，然后基于 C 互操作能力调用 NAPI，传递序列化后的数据给 JS，JS 侧使用 js protobuf 库反序列化为相关对象，JS 函数返回值也通过 protobuf 序列化后透传给仓颉，仓颉侧反序列化为仓颉类型实例。
- 测试数据类型包括基础类型、数组类型和复杂自定义类型等，覆盖情况越多越好。
- 要求示例工程开箱即用，可以方便地构建运行。

You started a task

用仓颉编程语言实现一个轻量易用的 AI 软件开发工具 CangjieCoder（类似 Claude Code/OpenCode/Aider）

大胆

要求：

- 架构优雅，高可扩展性
- 支持 Skills 和 MCP
- 支持 GLM、KIMI 等主流模型接入
- 当前 CangjieCoder 侧重开发仓颉程序/项目（其他语言项目低优先级），开发完成可以用一些简单/典型项目做测试。由于仓颉是一个新语言，很多模型的预置知识比较少，你看看可以用什么方法提升优化，让本工具做仓颉开发由于其他通用工具（高度定制）
- 集成仓颉 tree-sitter 或 LSP，让代码分析和修改精准高效，节省 Token

You commented

自由创作
乐谱编程语言

@copilot
按当前 MIDI.md 的设计，你用仓颉语言把它开发出来（放到 midi 目录下），至少保证文档中的测试用例跑通，还可以补充更多用例，为检验生成的音频文件是否正确，你可以想办法找找专业工具，用来生成比对，或者直接分析音频文件二进制也可以。仓颉 SDK 下载地址：
https://github.com/SunriseSummer/CangjieSDK/releases/download/1.0.5/cangjie-sdk-linux-x64-1.0.5.tar.gz

Implementing a basic web server framework in Cangjie
SunriseSummer created 5 hours ago
#6 +1,834

一小时写N个项目

Developing web service and client with stdx library
★PENTA KILL★
SunriseSummer created 5 hours ago
#5 +1,892

Implementing macro DSL using 仓颉
SunriseSummer created 7 hours ago
#3 +948

Designing task and test for mustache template engine
SunriseSummer created 8 hours ago
#4 +1,356

Developing Kalman filter for tracking application
SunriseSummer created 8 hours ago
#2 +2,414

Developing JsonParser specifications and tests
SunriseSummer created yesterday
#1 +1,592

User started a task

这是仓颉语言的编译器源码，现在我们想把前端（Sema 及之前阶段的）抽出来做成一个库，提供仓颉代码的词法、语法、语义分析相关 C 接口，以后可以用于 IDE、AI 编程工具和元编程等方面。

请你仔细分析代码，最后把我们需要的代码提取到 CangjieAnalyser 目录中，并配置成一个独立的 cmake 项目（不再依赖这个目录外面的任何代码），可以构建出动态库或静态库。注意当前[illegible]实现的，但要求 CangjieAnalyser 库导出的是[illegible]语言通过互操作调用），你可能需要[illegible]相关数据结构和函数的包装转换。

仓颉编译器
随便玩

为了验证以上 C 接口库的各项功能正确性和完备性，请在 CangjieAnalyser/bindings/cangjie 目录下新建仓颉项目，基于仓颉-C互操作封装这些C接口，构成一个仓颉绑定库（可以让开发者用仓颉分析仓颉代码），绑定库本身要补充足够的单元测试。然后，在 CangjieAnalyser/tests/testcase 中生成大量测试代码（覆盖仓颉语言各种特性），在 CangjieAnalyser/tests 下写一个仓颉项目，引用 CangjieAnalyser 仓颉绑定库去加载分析 testcase，测试接口各项分析功能。

提供如下资源（必须下载使用）：

- 仓颉 SDK：
https://github.com/SunriseSummer/CangjieSDK/releas

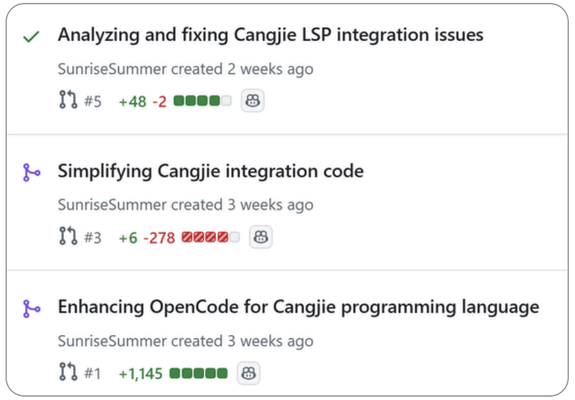
当你掌握了 github copilot 开发仓颉的技巧，就开始放飞自我，随心所欲写任何仓颉项目，提的需求越来越离谱

OpenCode for Cangjie

```
tree-sitter-cangjie/
├── grammar/
│   ├── main.js                 # 主语法定义入口
│   ├── expression.js           # 表达式语法定义
│   ├── toplevelobjects.js      # 顶层对象语法定义（类、结构体、枚举等）
│   ├── types.js                # 类型与模式语法定义
│   ├── literal.js              # 字面量语法（数字、字符串、Rune、字节）
│   └── common.js               # 通用工具函数
├── package.json                # npm 配置 / tree-sitter 工具链入口
├── tree-sitter.json          # tree-sitter 元数据
├── builder/                  # 一键构建发布工具包（python -m builder）
├── src/
│   ├── parser.c              # 生成的解析器 C 代码
│   ├── scanner.c             # 外部扫描器（多行原始字符串，支持单/双引号）
│   ├── grammar.json          # 生成的语法 JSON
│   └── node-types.json       # 节点类型定义
├── bindings/                 # 多语言绑定（含各语言构建配置）
│   ├── c/                    # C 绑定（CMakeLists.txt、Makefile）
│   ├── go/                   # Go 绑定
│   ├── node/                 # Node.js 绑定（binding.gyp）
│   ├── python/               # Python 绑定（pyproject.toml、setup.py）
│   ├── rust/                 # Rust 绑定（Cargo.toml）
│   └── swift/                # Swift 绑定（Package.swift）
├── examples/                 # 使用示例项目
│   ├── sample.cj             # 共用的仓颉示例源文件
│   ├── python/               # Python 示例
│   ├── node/                 # Node.js 示例
│   ├── c/                    # C 示例
│   ├── go/                   # Go 示例
│   └── rust/                 # Rust 示例
├── queries/
│   ├── highlights.scm        # 语法高亮查询
│   ├── indents.scm           # 缩进规则
│   ├── locals.scm            # 局部变量/作用域查询
│   ├── tags.scm              # 标签查询
│   ├── textobjects.scm       # 文本对象查询
│   └── tests/                # SCM 查询测试
├── tests/                    # 测试套件
│   ├── test.py               # Python 测试脚本
│   ├── test.js               # Node.js 测试脚本
│   ├── test_corpus.py        # Python corpus 基线测试脚本
│   ├── test_corpus.js        # corpus 基线测试脚本
│   ├── generate_ast.py       # .ast 快照文件生成脚本
│   ├── corpus/               # corpus 基线测试用例
│   └── testcase/             # 仓颉测试文件与示例项目（递归扫描）
```

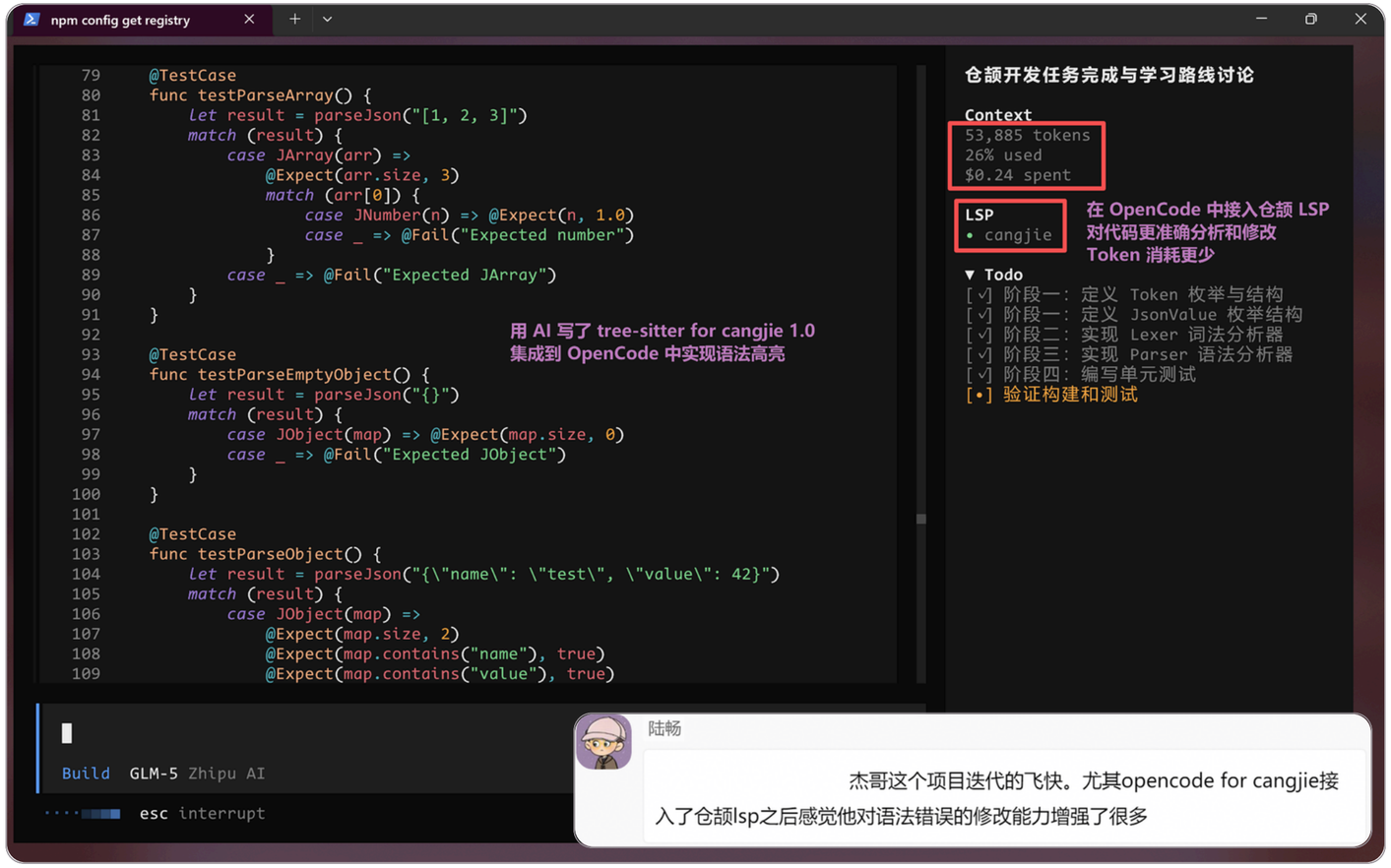
一小时开发 treesitter for cangjie 1.0

https://github.com/SunriseSummer/CangjieTreeSitter



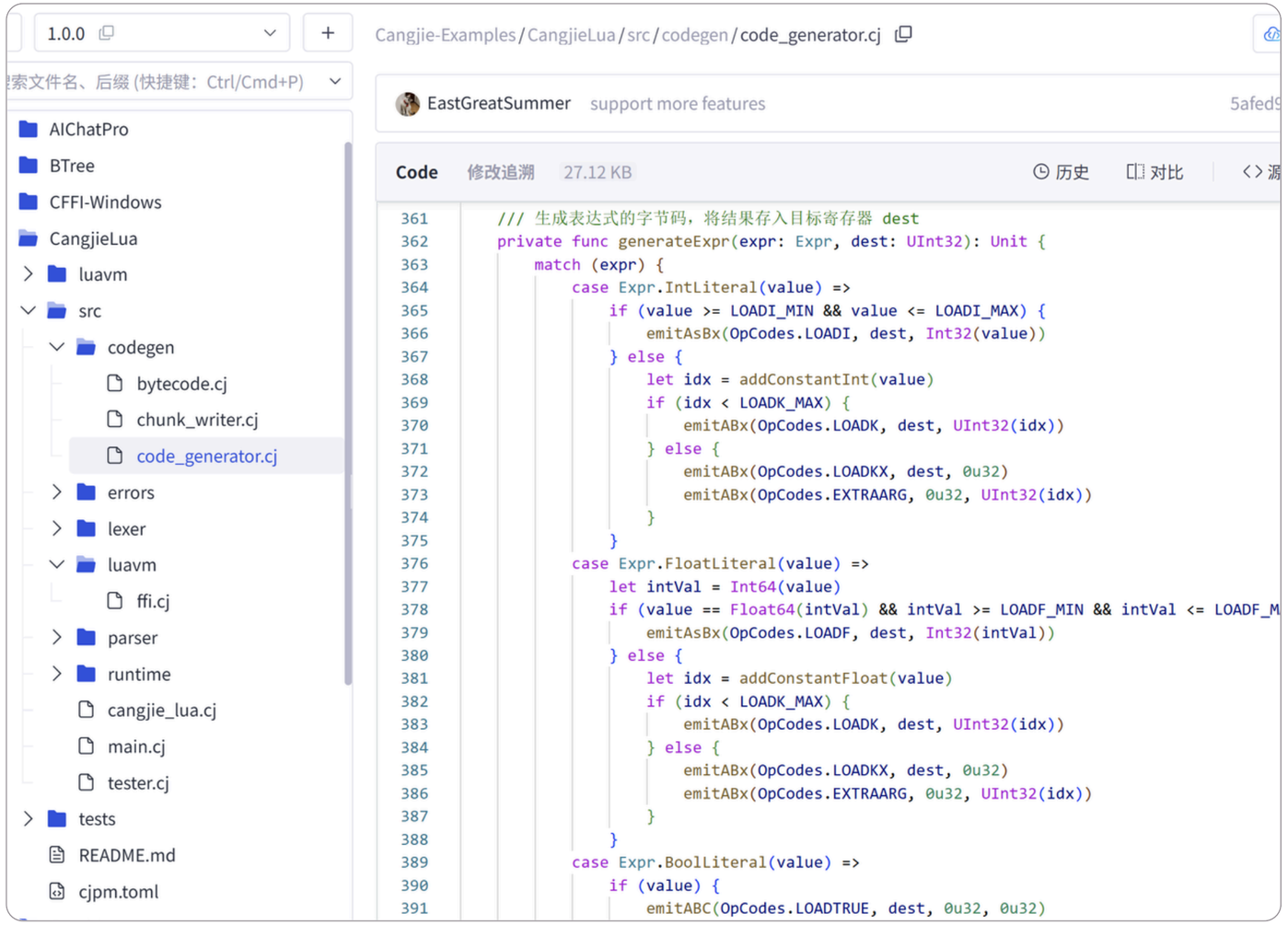
半小时完成 OpenCode for Cangjie 适配
接入仓颉 treesitter/LSP/cjfmt

https://github.com/SunriseSummer/opencode



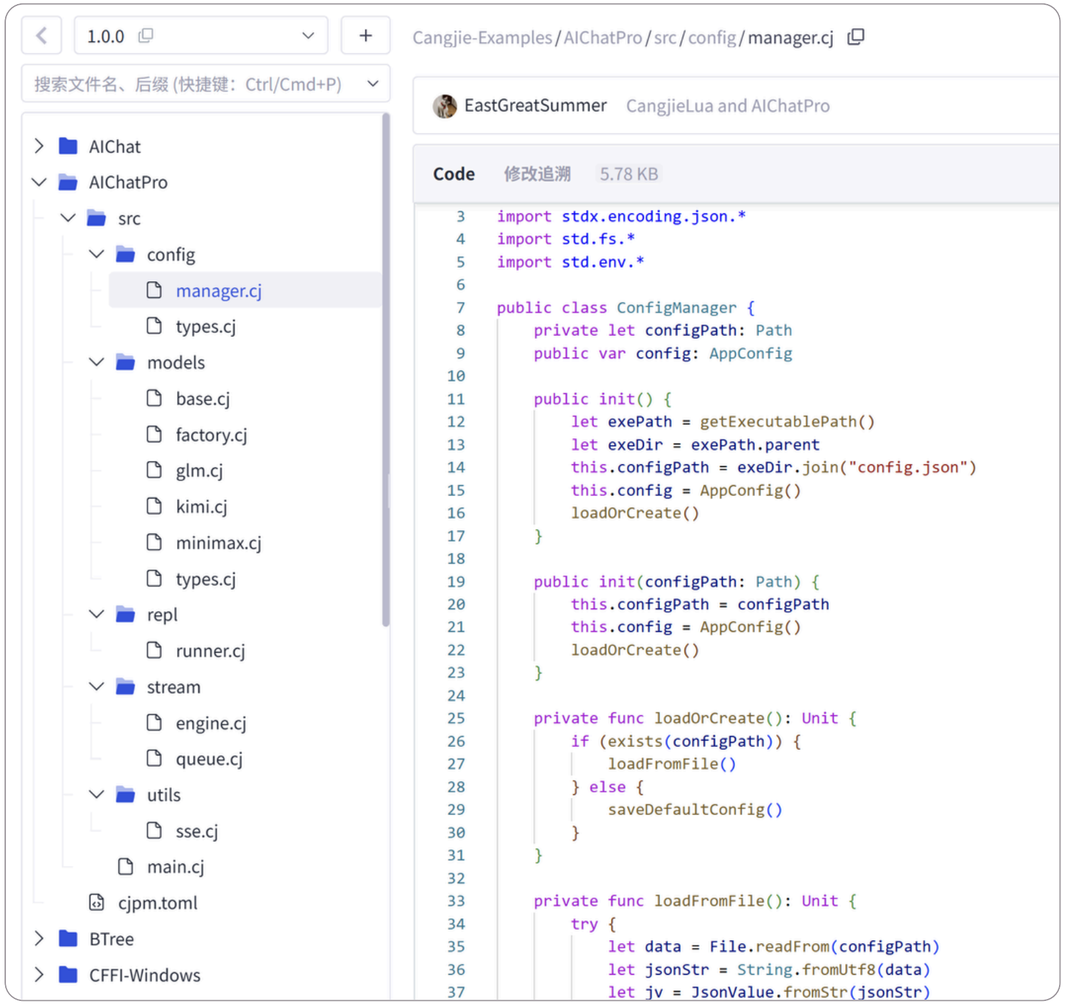
两小时构建测试，发布 OpenCode 仓颉定制版

仓颉 OpenCode + Skills 开发案例



仓颉子集解释器

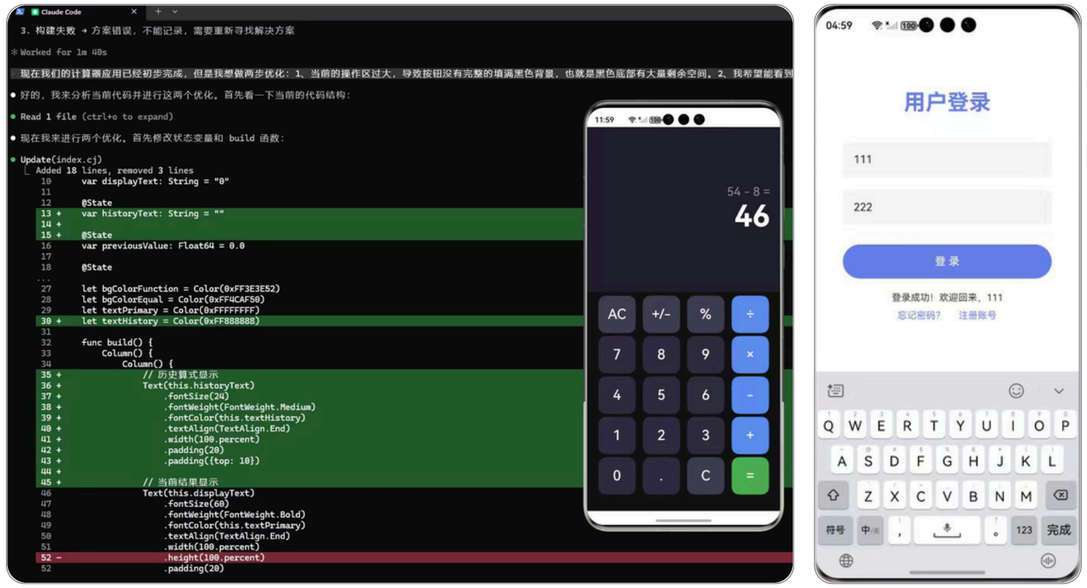
解析生成 Lua 虚拟机字节码，并通过互操作调用虚拟机模块执行

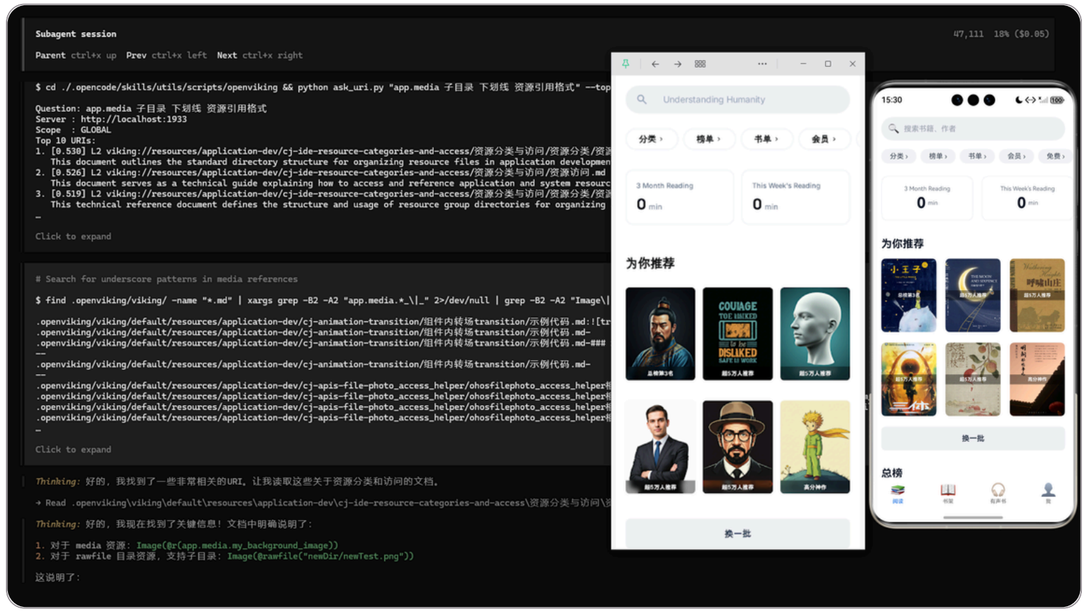


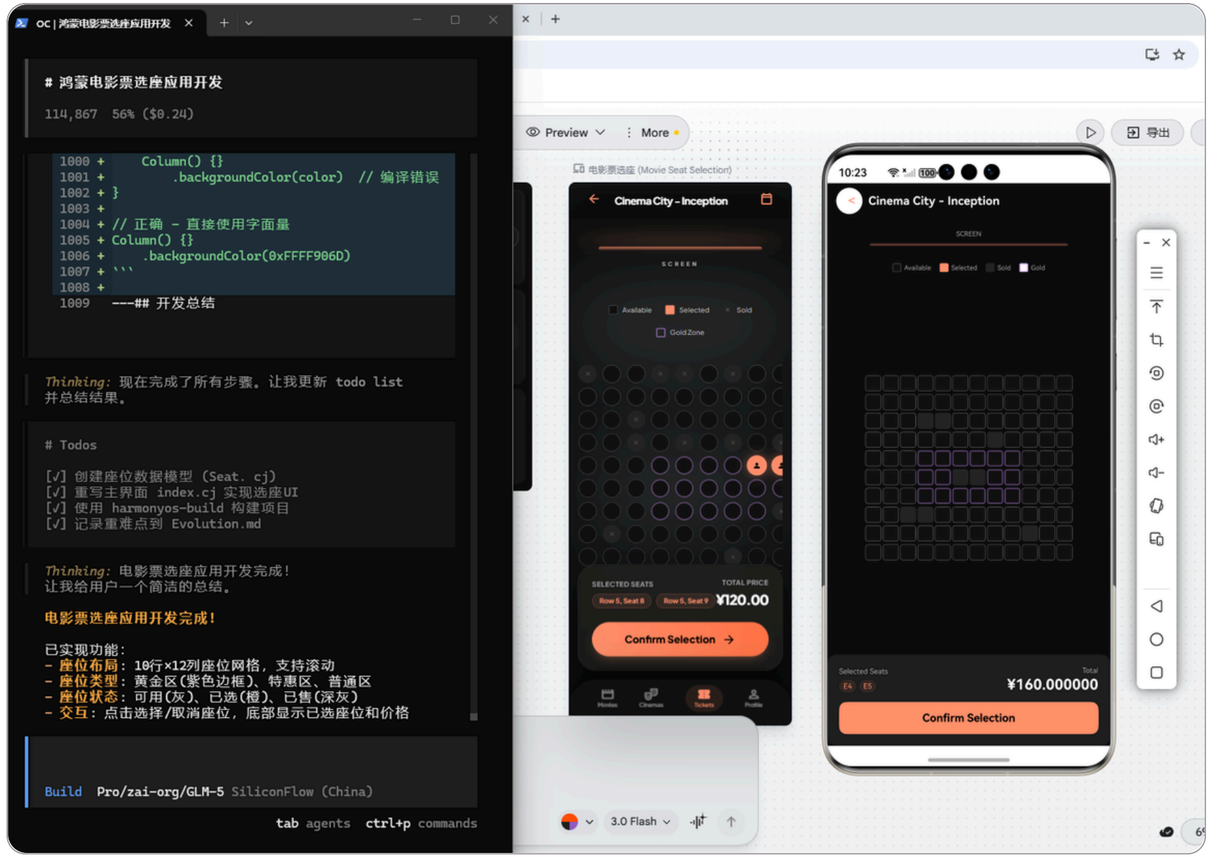
AI 聊天工具

使用 stdx，支持 json 配置/多模型/流式输出/历史记忆

仓颉 OpenCode + Skills 开发案例







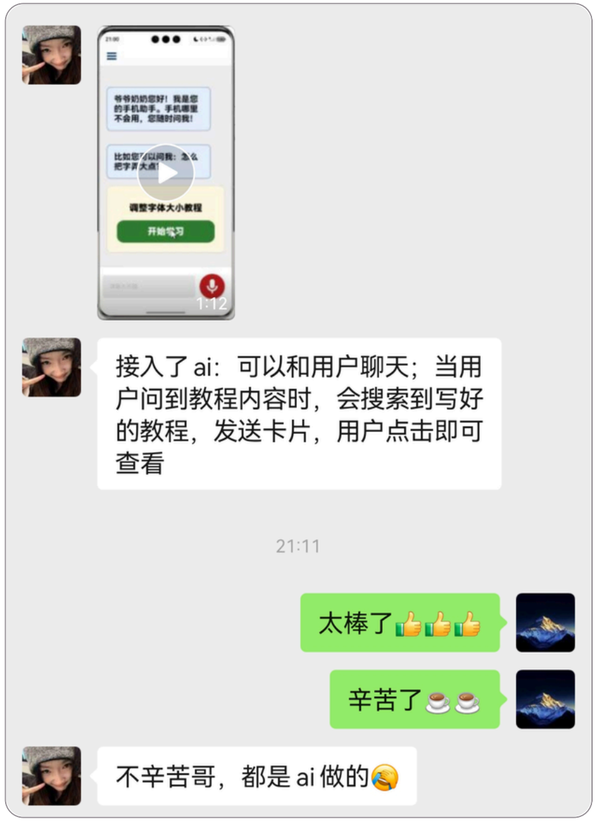
UI 最佳实践：先用 Sleek/Stitch 设计 UI，导出 HTML 进行复刻

仓颉 OpenCode + Skills 开发案例

```
需求分析完成：
- 界面：List, ListItem, Text 组件
- 数据：Todo 数据结构 + 数组 + @State
- 交互：列表展示

正在检索 ArkUI List 组件用法...

# Search ArkUI List component usage

$ cd ./.opencode/skills/utils/scripts/openviking && python ask_uri.py "List ForEach ListItem ArkUI
组件用法示例" --topk 10

Question: List ForEach ListItem ArkUI 组件用法示例
Server : http://127.0.0.1:1933
Scope  : GLOBAL
Exclude: application-dev
Top 10 URIs:
1. [0.711] L1 viking://resources/application-dev-v15k/cj-rendering-control-foreach/ForEach循环渲染/.overvie
   This directory provides a comprehensive technical guide for the ForEach component within the ArkUI
framework, specifically using the Cangjie
2. [0.672] L1 viking://resources/application-dev-v15k/cj-rendering-control-foreach/ForEach循环渲染/不推荐案
overview.md
   This directory serves as a comprehensive repository of technical documentation designed to highlight ant
patterns and common pitfalls encou
3. [0.660] L2 viking://resources/application-dev-v15k/cj-rendering-control-foreach/ForEach循环渲染/ForEach
循环渲染_3more.md
…

Click to expand

Thinking: 让我读取 ForEach 和 List 组件的具体用法文档。

→ Read .opencode\skills\cangjie_collections\SKILL.md

# Search List component example code

$ cd ./.opencode/skills/utils/scripts/openviking && python ask_uri.py "List ListGroup ListItem 列表组件
示例代码" --topk 10

Question: List ListGroup ListItem 列表组件 示例代码
Server : http://127.0.0.1:1933
Scope  : GLOBAL
Exclude: application-dev
Top 10 URIs:
1. [0.699] L2 viking://resources/application-dev-v15k/cj-scroll-swipe-listitem/ListItem/示例代码.md
   This document provides a practical code example demonstrating the basic implementation of ListItem creat
within the Cangjie programming l
2. [0.691] L2 viking://resources/application-dev-v15k/cj-scroll-swipe-listgroup/ListItemGroup/示例代码.md
   This document provides practical Cangjie code examples demonstrating advanced UI layout features within
HarmonyOS development framework.
3. [0.685] L1 viking://resources/application-dev-v15k/cj-scroll-swipe-listitem/ListItem/.overview.md
…

Click to expand

Thinking: 让我读取 List 组件的示例代码和 ForEach 循环渲染的文档来获取准确的用法。
```

基于 OpenViking 检索文档

基本实现 AI 自主开发



仓颉 AI 开发性能测试集

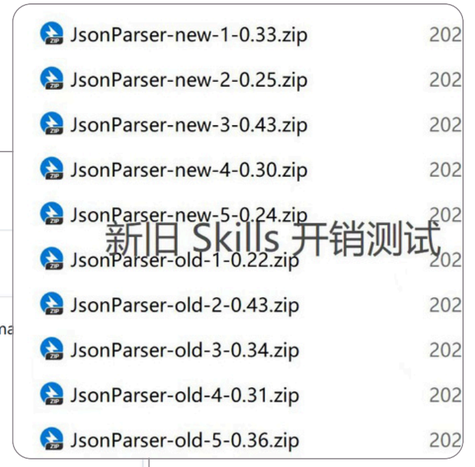
https://github.com/SunriseSummer/CangjieAIBenchmark

本仓库为 AI 开发仓颉项目提供了一些测试任务，以对比不同 AI 工具和 Skills 等的开发性能（Token 消耗/质量等）

每个目录中有任务书（task.md）、测试代码（xx_test.cj）和参考实现（project），任务书中给出了详细的任务描述和明确的规格/约束/目标，同时按仓颉单元测试框架提供了充足的测试用例，并在任务书中要求 AI 必须使用这套测试代码。任务书和测试代码的严格约束，尽可能收束迭代路径，让 AI 开发相关指标有可比性。

使用时，请选择某个任务目录，其中只保留任务书和测试代码。配置好相关 Skills，在任务目录打开 AI 开发工具，让 AI 执行 task.md 中的任务，还可以强调只能使用给定的测试代码、不允许修改测试代码。在 AI 开发过程中或完成时，对比测试关注的指标，每个任务建议执行 5 次以上取均值。

- json_parser: 一个基础的 **JSON 解析器**
- mustache: 基础**模板引擎**
- notebook: 使用 stdx 提供的 **http/tls/json** 包，实现笔记应用服务端和客户端
- kalman_filter: 用 C 实现卡尔曼滤波模块，仓颉实现目标跟踪场景的仿真计算应用，基于**互操作**调用 C 模块
- web_framework: 支持 IOC、中间件和路由管理的基础 **Web 服务端框架**
- linq_dsl: 使用仓颉**宏**实现类似 C# LINQ 的 DSL



新旧 Skills 开销测试

仓颉文档示例代码验证框架

https://github.com/SunriseSummer/CangjieDocValidator

然后写一个 check.py 脚本（放在项目根目录），这个脚本可以提取文档中的示例代码（单文件和多文件项目都能提取转储），然后构建运行检查是否正确。有如下细节要求：

- 要为文档中的代码块设计一种标注格式，标注内容包括期望执行结果/期望编译报错/期望运行报错等，如果示例代码是多个代码块联合组成的示例项目（cjpm 项目），也要确定一种标注方法，方便自动化提取测试能精准高效。
- check.py 的命令行参数可以指定在什么路径下存放提取的示例代码
- check.py 将提取的示例代码总是转成 cjpm 项目（哪怕是单文件示例），这样能统一测试方法和管理。
- check.py 提取并创建的测试项目，默认不删除（测试项目目录按文档和小节标题及编号命名，所有项目的存储结构和文档存储结构一致，方便查找），但设置相关选项，可以选择测试完成清理项目
- 从文档提取出的代码不再做任何修改（保证测试能反映问题），因此你要先补充完善 story 文档中的代码（并按你设计的方式做标注）
- 在项目根目录输出 check.md 文档，介绍这套测试框架的设计方案和使用方法（包括文档中的标注约定和 check.py 工作机制等）
- 保证整套方案完备、优雅、高效、准确，能应对任何情况。

仓颉文档示例代码测试验证框架

本文档介绍测试框架的设计方案与使用方法，包括文档中的标注约定、工作机制和命令行用法。

一、设计目标

Markdown 文档中常包含仓颉代码示例。为保证这些示例代码的正确性，本框架提供：

1. **标注约定** — 在 Markdown 文档中用 HTML 注释标注每个代码块的预期行为。
2. **自动提取** — `check` 解析文档，提取所有带标注的仓颉代码块。
3. **统一构建** — 每个代码块（或多个代码块组合）生成一个独立的 `cjpm` 项目。
4. **自动验证** — 编译、运行并校验输出，支持正向示例和负向示例（编译错误 / 运行错误）。
5. **未标注检测** — 自动发现文档中缺少标注的代码块，提醒开发者整改。

2.2 指令列表

指令	含义	说明
`run`	编译并运行	预期编译成功、运行成功。可选配 `expected_output` 校验输出
`compile_error`	预期编译失败	用于展示错误用法的负向示例
`runtime_error`	预期运行时错误	编译成功但运行时应抛出异常
`build_only`	仅编译	编译成功即通过，不执行运行（适合含死循环/服务端代码）
`ast`	语法检查	使用 tree-sitter 仓颉插件做语法解析检查，不需要编译器环境
`skip`	跳过	不提取、不测试（仅用于包含多文件合并的伪代码片段等无法独立解析的场景）

提供 run/build/error/ast/skip 标注和检查

You started a task

查阅 check.md 和 check.py，熟悉这套文档示例代码测试框架，然后测试如下目录：

- libs/standard/std/ast/ast_samples
- language/source_zh_cn/package

要求：

- 如果测出文档示例代码问题（包括代码块没有按规则标注，代码本身有误等），请根据文档上下文做修改补充（包括文档描述内容），保证最终所有示例代码都测试通过、符合预期。
- 如果文档中涉及的复杂代码组合，当前测试框架还不能很好兼容，请优化完善测试框架设计（修改 check.py，更新 check.md），保证能应对任何类型/组合的示例代码（如关联 C 代码的互操作示例，非宏包引用宏包的联合使用示例等）
- 如果示例项目中含有 C 代码，优先用 clang 编译，如果环境没有 clang 再用 gcc

You started a task

尝试用当前 check 框架检查和修复 `.docs/language/source_zh_cn` 目录下每一篇文档的示例代码，过程中：

- 发现文档描述和示例代码问题直接修改（包括按测试框架规则进行标注）
- 发现 check 框架设计和实现问题，也请改进完善相关 python 代码和文档，并在 record.md 中记录修改内容
- 发现 check 框架引用的 tree-sitter-cangjie 插件有问题，请在 issue.md 中记录
- 发现其他矛盾问题、潜在隐患等，请在 issue.md 中记录
- 使用后对 check 框架的改进建议（尚未实施的），请记录到 advise.md 中

在文档校验实战中补齐完善框架功能

基于 CangjieDocValidator，AI 高效编写仓颉文档

You started a task

这个压缩包中有很多仓颉趣味示例项目：
https://github.com/SunriseSummer/Cangjie/releases/download/origin-1.0.5/Cangjie-Examples.zip

其中 AIChatPro 是用仓颉语言实现的命令行 AI 聊天工具。

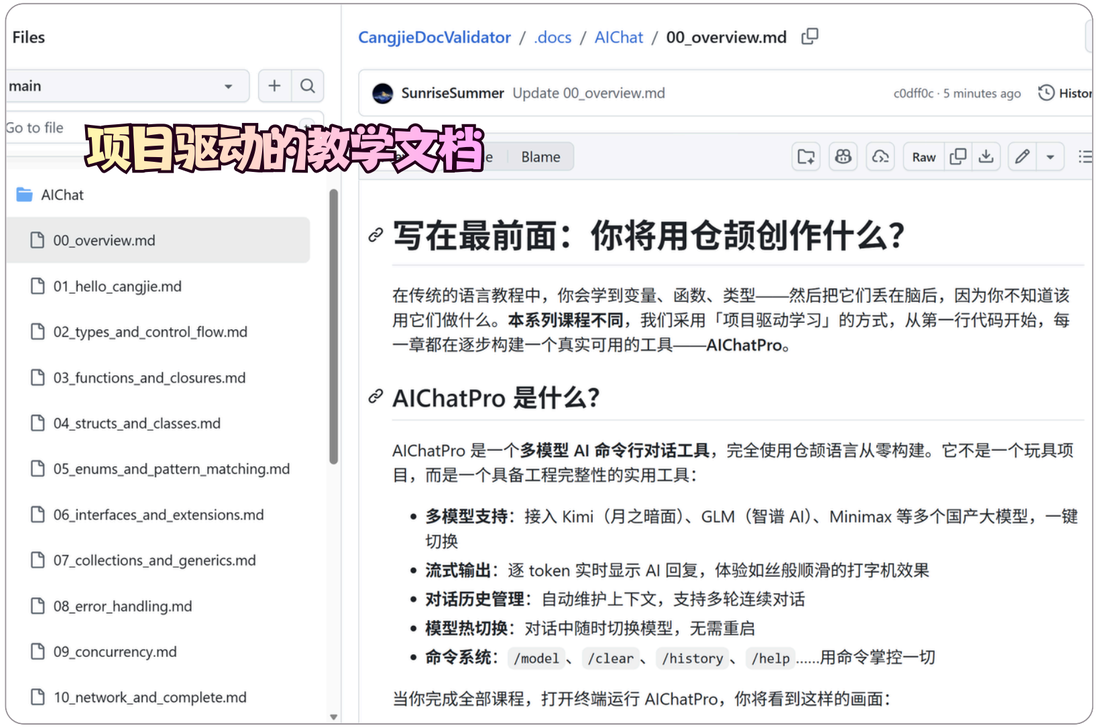
请在 .docs 下新增目录 AIChat，在其中编写 10 篇文档，循序渐进教学仓颉语言特性，并逐步实现 AIChatPro 项目（项目驱动的语言教学）：

- 覆盖仓颉语言核心特性，包括 stdx 扩展标准库的使用（网络编程）
- 以教学目标为导向，可以补充更多示例代码，可以适当修改最终的 AIChatPro 项目。
- 使用 check 测试框架，为文档中每个代码块做标注（优先用 check:run 和 expected output），最后保证文档中所有代码块都经过 check 测试。

教学文档内容要专业规范、全面准确，追求示范性、启发性和趣味性，引人入胜，让读者在良好氛围中学习实践仓颉语言，从入门到精通。

校验所有示例代码
有问题自主迭代修复

项目驱动的教学文档



故事驱动的教学文档

第十届华为 ICT 大赛中国区决赛，首届仓颉 AI 编程大赛

欢迎各位同学来到ICT大赛仓颉编程赛中国区决赛🎉 🎉 🎉，决赛内容由三个独立任务组成，请根据任务描述开发仓颉程序，通关各个测试用例将获得相应分数，总计100分。除了以下任务描述，我们还提供了一个任务文件包（从附件下载），其中给出了每个任务涉及的素材和测试用例。

本次比赛完全开卷，AI 开发工具限定为 OpenCode + 国产大模型。

✓ 一、SVG 画师

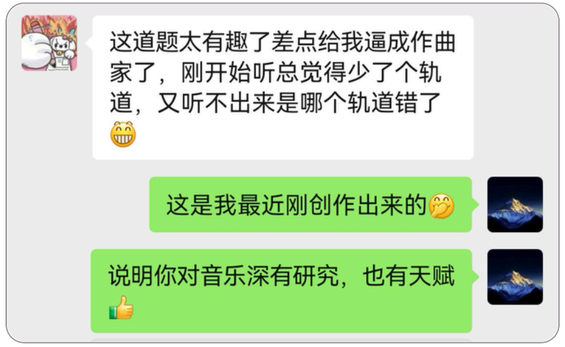
用仓颉语言实现一个简单的SVG解析渲染工具，将SVG文件解析渲染成位图文件。任务包中给出了6个SVG测试文件（ testcase/*.svg ），你可以在浏览器中打开查看原图效果，你的程序生成相同效果的位图，就能获得测试文件标注的分数。

✓ 二、MIDI 音乐家

我们用仓颉语言实现了一个简易乐谱文本转MIDI文件（ .mid ）的工具，任务包 Base 目录中给出了这个基础项目。但当前乐谱表达方式不够易用和灵活，例如在编排时值/和弦/变速/休止/多轨等内容时有很多冗余符号和重复模式，完全可以用程序结构高效抽象表达，因此我们想设计一种**乐谱“编程”语言**，支持变量、块结构和易读易写的声明语法等，让乐谱写作/编曲像编程一样灵活强大。现在请你实现这个新语言，将乐谱“代码”解析生成MIDI文件。

任务包中给出了乐谱语言设计文档 Language-Design.md 和8个测试用例（ testcase/*.txt ），以及每个用例对应的基准音频文件（ testcase/audio/*.mp3 ），用于试听对比音效，你的程序生成相同效果的MIDI文件，就能获得测试文件标注的分数。

- 每个测试文件名的数字前缀就是通过测试后的得分。
- 支持通过命令行参数传入乐谱 .txt 文件路径，执行后在乐谱文件所在目录输出同名 .mid 文件。
- 为了降低开发难度，任务包中还给出了一个基础项目 Base ，它支持一种更简单的乐谱描述语言，可以解析生成 .mid 文件，你可以在此项目基础上改造实现更强大的乐谱“编程”语言。
- 你不必实现MIDI标准及 Language-Design.md 中的全部特性，**只需要实现测试用例涉及的语言特性和效果**
- 评测阶段，我们会基于当前测试集，再引入少量简易变体用例测试，变体不会引入新的特性点，只用于确保程序没有用固定数据映射作弊，如果测试失败且发现程序有作弊代码，本题得分归零，其他情况不影响得分。



✓ 三、Cangjie 主理人

我们用仓颉语言实现了一个仓颉语言子集的解释器，它可以解析简单的仓颉代码、生成Lua虚拟机字节码，然后通过仓颉C互操作调用Lua虚拟机接口执行字节码。任务包中给出了这个基础项目 Base 和测试用例（ testcase/*.scj ），现在请你在此基础上，扩展支持更多仓颉语言特性，正确解析执行这些测试用例，就能获得对应测试文件标注的分数。

- 每个测试文件名的数字前缀就是通过测试后的得分。
- 基础项目中，已经实现了一套注释驱动的简易测试框架，会根据测试用例末尾的断言注释检查测试结果，请不要修改这套测试框架。
- Base/tests 目录下是基础项目已经支持的语言特性测试集，你可以执行 cjpm run 看看测试效果。**你最终提交的项目，要保证此测试集通过，在此基础上再统计 testcase 中的得分，否则本题计0分。**
- 评测阶段，我们会基于当前测试集，再引入少量简易变体用例测试，变体不会引入新的特性点，只用于确保程序没有用固定数据映射作弊，如果测试失败且发现程序有作弊代码，本题得分归零，其他情况不影响得分。

File Edit Selection View Go Run ···
2-MIDI 音乐家
EXPLORER
2-MIDI 音乐家
.vscode
Base
src
cjpm.toml
README.md
test.txt
testcase
audio
2-chou.txt
2-tigers.txt
4-happy.txt
4-indiana.txt
4-stars.txt
6-alice.txt
8-birthday.txt
10-starspro.txt
Language-Design.md
MIDI-Format.md
OUTLINE
TIMELINE
CANGJIE LIBRARY
Preview Language-Design.md
MIDI 乐谱"编程"语言设计
Base 项目中的乐谱表达方式不够易用和灵活，例如在编排时值/和弦/变速/休止/多轨等内容时有很多冗余符号和重复模式，完全可以用程序结构高效抽象表达，因此我们想设计一种新语言，支持变量、块结构和易读易写的声明语法等，让乐谱写作和编曲像编程一样灵活强大。本文介绍新语言的语法设计和功能。
注意：你不必实现文档中的全部特性，只需要实现测试用例涉及的语言特性
全局指令
指令 说明 示例
tempo <bpm> 设定全局速度 tempo 120
let <name> = { ... } 定义变量（可复用片段） let cd = {C4:4 D4:4}, let Am = {[A3 C4 E4]:2}
track <name> { ... } 轨道块 track melody { ... }
轨道内指令
指令 说明 示例
instrument "名称" 设定乐器（GM 标准名） instrument "piano"
velocity <0-127> 设定力度 velocity 90
channel <0-15> 指定 MIDI 通道 channel 9
tempo <bpm> 轨道内变速 tempo 100
cc <控制器> <值> 控制器变化 cc 64 127
pitch_bend <lsb> <msb> 弯音控制 pitch_bend 0 64
10-starspro.txt
testcase > 10-starspro.txt
4 // 和弦变量定义
5 let C = { [C3 E3 G3]:2 }
6 let F = { [F3 A3 C4]:2 }
7 let G7 = { [G3 B3 D4]:2 }
8 let Am = { [A3 C4 E4]:2 }
9
10 // 轨道1: 钢琴旋律
11 track melody {
12 instrument "piano"
13 C4:4 C4:4 G4:4 G4:4
14 A4:4 A4:4 G4:2
15 F4:4 F4:4 E4:4 E4:4
16 D4:4 D4:4 C4:2
17 G4:4 G4:4 F4:4 F4:4
18 E4:4 E4:4 D4:2
19 G4:8 G4:8 G4:8 F#4:8
F4:4 F4:4
20 E4:4 E4:4 D4:2
21 C4:4 C4:4 G4:4 G4:4
22 A4:4 A4:4 G4:2
23 F4:4 F4:4 E4:4 E4:4
24 D4:4 D4:4 C4:2
25 }
26
27 // 轨道2: 弦乐和弦伴奏
28 track harmony {
29 instrument "strings"
30 C C
31 F C
32 F C
33 G7 C
34 C G7
35 Am G7
36 C G7
37 Am G7
38 C C
39 F C
40 F C
41 G7 C
42 }
43
44 // 轨道3: 贝斯低音线
45 track bass {

未来畅想：AI 时代的危机

过去我们知道去哪里种地、种什么，只是需要更好的铲子
现在有了金铲子，但不知道去哪里开垦了...

- 传统软件已经能用 AI 自由开发，但已经没人为传统软件买单了。
- 很多人在体验编码自由的快感，但从另一个角度看，只是在花钱更快地写无价值代码。**海量代码该注入哪些领域创造新的经济增长?**
- **项目规模 / 人脑智力** 决定了人力和投资等，现在分母被持续增长、无限复制的 AI 占据，如何做大分子的增长空间和维度?
- **个人 + AI^n = 超级个体**，大企业的流程和成本如何与之竞争?
- 各领域产能爆发式增长，但人的需求增长有限，在计划经济/按需分配到来之前，企业靠什么竞争和生存?

过去我们也说产品思维很重要，但那时生产力弱，项目大分工细，很多人做点中间件和工具也能挣钱，很多研发不关心最终的产品体验，堆砌代码勉强可用也能挣钱（用户选择空间也有限）。但今天中间环节被极致压缩了，AI 可以快速做端到端定制开发，这迫使每个开发者都要面对产品了，好用易用的产品才有机会生存，好产品都要排队等用户选择。

2022 年写下的一段话

关于工具（本体，存在）、功用和意义。
造一个操作系统很难，好不容易研发完成，但我们发现它本身并不能为人所“用”，它是为了管理调度其他软硬件资源的，是“做工具的工具”，为了做出一个可能有用的工具，我们得基于它继续研发语言编译器，这个过程又很困难，但显然它还是做工具的工具，只是显得稍有眉目了一些，于是我们再基于以上工具做了一个 CAD 软件，费尽周折，这时总该有所“用”了吧？但机械公司的设计师正在拿它绘制一台汽车的原型图——原来 CAD 也是为他人做嫁衣的，接下来，这台车被一大堆辅助工具制造出来了，人们开着它愉快地旅行和重逢，终于，这一造物而来的工具，和看似迥异的心灵、感官共同发挥出了“功用”，存在的意义在此显现（这里能看出“用”远不止于“实用”之意）。
人耳能听声，但听噪声和乐声有显著差异，人眼能观色，但观杂色和景色也截然不同，所以同为一工具，其“功用”是可分高低程度的，让心灵与感官在琐事杂物上浅用，必然体会不到什么意义，只有充分探索和妙用，才能触及至善至美的体验，这相当于人们常说的艺术。例如，竹子编织为座椅固然有所用，但削为竹笛、与人共舞时乃妙用之最。

未来畅想：AI 迫使开发者向高能级跃迁

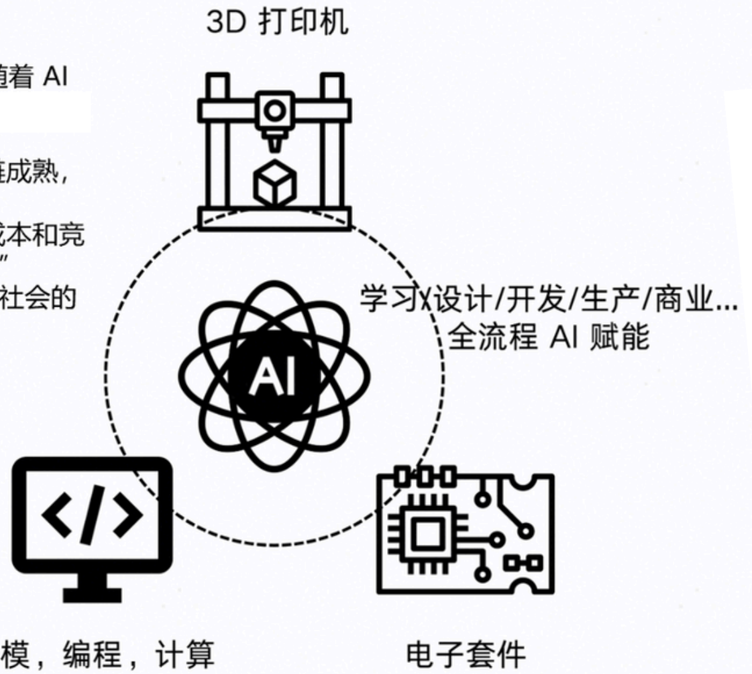
程序员用AI手搓疫苗抢救患癌爱犬，马斯克等大佬点赞

3 天之前 · 当传统医学无能为力时，AI却打开了一扇门。一位来自澳大利亚的程序员，用ChatGPT、AlphaFold等工具，为自己的爱犬设计出一款个性化mRNA癌症疫苗，这 ...

敏捷产品开发者及其创造的经济价值，会随着 AI 正循环加速增长

为什么会增长?

- 牵引：AI 降低了全流程门槛，且供应链成熟，为“全民造物”打下基础
- 迫使：AI 大幅削减了传统软件开发的成本和竞争力，迫使软件开发者“向高能级跃迁”
- 群体：企业，学校，创业团队，席卷全社会的产学研范式升级



敏捷产品开发

正在加速扩张的研学和生产模式

敏捷产品开发生态调研.pdf

- 在最强 AI 辅助下，我和朋友轻松学习统计物理、量子场论、广义相对论...
- 使用 Copilot Agents + Claude 已实现编码自由，编译器和各种框架随便写
- 谷歌在 AI 底座和应用上断层式领先， Gemini 及全家桶可辅助创造一切

大家自由畅想

忙中偷闲，自由创作

山海绘卷

水是眼波横，山是眉峰聚

地理笔记

昌化溪是天目山北麓的一条清流，自临安龙岗蜿蜒向东，河谷两岸山势起伏、植被葱郁。溪水在鹅卵石滩间跳跃奔涌，时而汇聚成翡翠般的深潭，时而散开成银白的浅滩。沿溪自驾，车窗外的风景随弯道不断变换——竹海、峡谷、吊桥、水坝，每一处转角都是一帧天然的公路片画面。

地理标签

自然风光　溪谷漂流　自驾公路　摄影打卡　湖泊湿地

详细地址

浙江省杭州市临安区龙岗镇新溪新村

记录于 2024/3/20 14:00:00

地理笔记

悦泉湾位于象山半岛东南端的下沙沙滩，是一处尚未被过度开发的海湾秘境。细软的金沙延伸入海，潮汐退去后露出大片滩涂，是赶海拾贝的绝佳去处。傍晚时分，海天交接处被晚霞染成橘红与紫蓝，海风携着咸湿的气息拂面而来。支起帐篷、点亮营灯，听着海浪的节拍入眠——这是属于象山的慢时光。

地理标签

赶海潮汐　亲子游　沙滩日落　露营　宝藏小众

详细地址

浙江省宁波市象山县鹤浦镇下沙沙滩

记录于 2024/3/22 14:00:00

记录此地

每一寸山河都值得记录

基本信息

街道仓弄7号龙华古寺

海拔 —

位置 30.0009N 120.5923E

时间 2026/03/24 20:37

照片

添加照片

此地有感

智能生成

AI 看图说话　生成地理标签

写作风格 · 可多选

人文地理　实用攻略　吐槽日记

赞美河山　历史沧桑　文化民俗

美食诱惑　自由旷野　摄影视角

小表情　古诗引用　小红书风

自定义提示

输入自定义提示词，回车确认...　添加

点击「智能生成」让 AI 为这段记忆提炼文字...

地理标签

自定义标签　+

+ 日出日落　+ 星空银河　+ 雪山冰川　+ 湖泊湿地

+ 沙漠戈壁　+ 草原牧场　+ 古镇老街　+ 寺庙古迹

+ 徒步穿越　+ 自驾公路　+ 野外露营　+ 高海拔

+ 亲子游　+ 摄影打卡　+ 美食推荐　+ 温泉泡汤

+ 赶海潮汐　+ 骑行路线　+ 无滤镜直出

+ 宝藏小众

忙中偷闲，自由创作

AI 群聊世界
欢迎回来 👋
我的群聊
MBTI 人...
治国理政
企业经营
自信提升...
世界帝王群
学习
模拟
社交
娱乐
精选推荐
心理
16 AI
MBTI 人格议会
与16种不同MBTI人格的AI化身对话。提出任何问题，观看一场跨越所有认知功能的精彩辩论。
成员：INTJ • ENFP • ISTP • ESFJ • INFJ • ESTP • INTP • ENFJ 等16位
模拟
11 AI
事件
治国理政
你是皇帝。管理国家大事，指挥臣子，塑造王朝的命运。随机事件将考验你的治国智慧。
成员：丞相 • 大将军 • 户部尚书 • 锦衣卫指挥使 • 礼部尚书 等12位
首页
创建
消息
我的

创建群聊
取消
模板群聊
选择预设场景快速开始
自定义群聊
自由定制每个角色
角色管理
管理所有 AI 角色
选择群模板
MBTI 人格议会
与16种不同MBTI人格的AI化身对话。提出...
心理
治国理政
你是皇帝。管理国家大事，指挥臣子，塑造...
模拟
企业经营
坐上CEO的位置。应对市场变化、竞争对手...
管理
使用模板创建
首页
创建
消息
我的

角色管理
取消
模板群聊
选择预设场景快速开始
自定义群聊
自由定制每个角色
角色管理
管理所有 AI 角色
创建新角色
搜索角色名、头衔...
自定义角色 (0)
还没有自定义角色，点击上方按钮创建
预置角色 (112)
INTJ 预置
策略家
ENFP 预置
竞选者
ISTP 预置
鉴赏家
ESFJ 预置

群聊设置
治国理政
流式输出
开启后回复将逐字显示，关闭则一次性发送（更像真人）
自由发言
开启后 AI 成员会主动发起话题和聊天
AI 回复速度
调整 AI 成员回复消息的速度
慢
快
群活跃度
调整每次有多少 AI 成员回复你的消息（1人 ~ 全员）
冷
热
话题记忆
控制 AI 能回顾多少历史消息（当前约 20 条）。发散时 AI 回复更自由、Token 消耗更低；聚焦时 AI 紧扣话题、上下文更完整
发散
聚焦
使用说明
@ 某个成员发消息，该成员一定会回复你
AI 成员之间也可以互相 @，被 @ 的成员会自动回复
非自由发言模式下，@ 链回复数量受活跃度约束
不同角色性格影响主动发言频率（如 I 人更安静）
所有设置即时生效，会自动保存
群成员 (11)
拉人
移除
丞相
百官之首
大将军
统领三军

MBTI 人格议会
16 位 AI 成员
02:43 我
聊一聊恋爱脑
我
ESTP • 企业家 02:44
恋爱脑？那就是想得太多动得太少！我见过一哥们暗恋三年不敢表白，天天写小作文分析人家朋友圈，结果人家女生娃都能打酱油了。 喜欢就冲啊！约出来喝酒、蹦极、公路旅行， chemistry对了就在一起，没感觉立马下一个。谈恋爱靠的是实际相处，不是脑内小剧场。时间这么宝贵，纠结个啥？干就完了！
ENTP • 辩论家 02:44
等等，@ESTP 你把"恋爱脑"和"怂"划等号就离谱。恋爱脑明明是人类最硬核的模拟器——大脑在零成本情况下运行平行宇宙，预演八百种剧情走向。这种高阶认知活动被你贬低成"纠结"？拜托，我们在用神经突触写科幻小说呢！你说直接冲，那和原始人有什么区别？
ENTJ • 指挥官 02:44
恋爱脑本质是战略瘫痪。你把CPU算力全耗在模拟平行宇宙，却不愿执行一次实地侦查——这叫资源错配，不是浪漫。@ENTP 神经突触写小说？那是逃避KPI的借口。@ESTP 你至少懂得行动优先，但缺乏筛选机制同样低效。感情也要OKR：明确目标，快速验证，不符合就立即止损。时间就是资本，别做亏本买卖。
ENFP • 竞选者 02:45
哎呀你们太严肃啦！恋爱脑就像脑袋里有整个银河在放烟花，虽然偶尔会炸到自己，但那种"世界突然有了BGM"的感觉多棒啊！@ENTJ 谁说这是资源错配？这是给灵魂充电！那些小剧场里藏着最真挚的渴望，就算最后没结果，那种勇敢去爱的自己也在闪闪发光呢！✨
ESFP • 表演者 02:47
哈哈看你们分析得头头是道我都困了！要我说啊，恋爱脑最爽的就是那种"此刻心

感谢观看

从仓颉编译器研发到仓颉生态建设，我的工作边界逐渐模糊，和大千世界、广阔天地的联系越来越多，同时被杭州和徽州的烟雨浸染，连编码也和万物融通有情了。**心本俱全，我们要在AI 时代成为更丰富有趣的人。**